Report on Student Performance

Isaac Sinaga

2025-08-16

Introduction

In this report we will be studying the secondary education of two Portuguese schools. The dataset we will be using comes from the UC Irvine Machine Learning Repository. It was created by Paulo Cortez, and donated to UCI on Nov. 26, 2014. The dataset is licensed under a Creative Commons Attribution 4.0 International (CC by 4.0) license. For more information on sources, please refer to the References page. All inspired code will either be cited in code comments and/or on the References page.

Our target for this project will be attempting to predict the final year math scores of students at the two Portuguese schools, which is called G3. According to the Dataset Information on the UCI Machine Learning Repository website, the G3 score is the final year grade, which is issued at the 3rd period. G1 and G2 signify scores at the 1st and 2nd periods, respectively. All scores are graded from a score of 0 to 20. We will use many predictors from the dataset in order to try and predict the G3 scores. We will first load the data, analyze it, and then create three different models with different predictors to find out which one produces the most accurate result by calculating the RMSE.

The code and analysis shown here is all original. Very minimal and minor inspiration has been taken from knowledge and insights gained from resources such as the UCI Machine Learning Repository, the HarvardX Data Science Program, and GitHub Copilot. Nothing has been copy and pasted and has been written originally. All sections of code or analysis with inspiration has been cited properly in code comments or explanations. For more information about sources go to the References page.

Preparing the Data

To first begin our project, we will need to download the code from the UCI Machine Learning Repository. The following code will install and load all needed packages to do so:

```
## THE FOLLOWING DATASET WAS FOUND ON THE UC IRVINE MACHINE LEARNING REPOSITORY
## DONATED TO UC IRVINE ON 11/26/2014
## URL FOR ACCESS: https://archive.ics.uci.edu/dataset/320/student+performance
## ACCESS DATE: 8/11/2025
## CREATED BY PAULO CORTEZ
## LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION 4.0 INTERNATIONAL
## (CC BY 4.0) LICENSE
## DOWNLOAD LINK: https://archive.ics.uci.edu/static/public/320/student+performance.zip

## The following code that installs the required packages was inspired from code in the HarvardX Data S
if(!require(tidyverse)) install.packages("tidyverse", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: tidyverse
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
if(!require(caret)) install.packages("caret", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: caret
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
if(!require(readxl)) install.packages("readxl", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: readxl
```

```r
if(!require(readr)) install.packages("readr", repos = "http://cran.us.r-project.org")
if(!require(dplyr)) install.packages("dplyr", repos = "http://cran.us.r-project.org")
if(!require(randomForest)) install.packages("randomForest", repos = "https://cran.us.r-project.org")
```

```
## Loading required package: randomForest
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
if(!require(ggplot2)) install.packages("ggplot2", repos = "http://cran.us.r-project.org")
if(!require(corrplot)) install.packages("corrplot", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: corrplot
## corrplot 0.95 loaded
```

```r
if(!require(writexl)) install.packages("writexl", repos = "http://cran.us.r-project.org")
```

```
## Loading required package: writexl
```

```r
if(!require(utils)) install.packages("utils", repos = "http://cran.us.r-project.org")

library(tidyverse)
library(caret)
library(dplyr)
library(readr)
library(readxl)
library(writexl)
library(randomForest)
library(ggplot2)
library(corrplot)
library(utils)
```

Lines that contain 'if(!require)' were written from inspiration from the HarvardX Data Science program.

Now that all packages are automatically installed and loaded by our code, we will download the file from the UCI Machine Learning Repository. After downloading the dataset, the file contains data on both scores in Portuguese language and math, but we are only interested in the math data, so we will only extract 'student-mat.csv'. For simplicity and formatting purposes, we will also convert the CSV file into an Excel file. We can do that with the following code:

```r
url <- "https://archive.ics.uci.edu/static/public/320/student+performance.zip"
temp_dir <- tempdir()
outer_zip <- file.path(temp_dir, "student+performance.zip") ## this line has minor inspiration from Gitl
download.file(url, destfile = outer_zip, mode = "wb")

unzip(outer_zip, exdir = temp_dir)

student_zip <- list.files(temp_dir, pattern = "student.zip$", recursive = TRUE, full.names = TRUE)

unzip(student_zip, exdir = temp_dir)

csv_file <- list.files(temp_dir, pattern = "student-mat.csv$", recursive = TRUE, full.names = TRUE)

student_data <- read_csv2(csv_file)
```

```
## i Using "','" as decimal and "'.'" as grouping mark. Use `read_delim()` for more control.
```

```
## Rows: 395 Columns: 33
## -- Column specification --------------------------------------------------------
## Delimiter: ";"
## chr (17): school, sex, address, famsize, Pstatus, Mjob, Fjob, reason, guardi...
## dbl (16): age, Medu, Fedu, traveltime, studytime, failures, famrel, freetime...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
write_xlsx(student_data, path = "student-mat.xlsx")

df <- read_excel("student-mat.xlsx")
## DATASET WAS ACCESSED FROM URL: https://archive.ics.uci.edu/dataset/320/student+performance
## DOWNLOAD LINK: https://archive.ics.uci.edu/static/public/320/student+performance.zip
```

Now we have our dataset, and it is easy to read since it is in Excel format.

Data Cleaning

Before continuing our project, we need to clean the data.

We will begin by checking for NA values as those are always tricky to deal with.

```r
## Exploratory Data Analysis
## START CLEANING THE DATA SET
## Check for NA's
sum(is.na(df)) ## There are no NA values
```

```
## [1] 0
```

We can see that there are no NA values within this dataset. We will continue by examining the structure of the dataset now.

```r
## Examine the dataset
str(df)
```

```
## tibble [395 x 33] (S3: tbl_df/tbl/data.frame)
##  $ school    : chr [1:395] "GP" "GP" "GP" "GP" ...
##  $ sex       : chr [1:395] "F" "F" "F" "F" ...
##  $ age       : num [1:395] 18 17 15 15 16 16 16 17 15 15 ...
##  $ address   : chr [1:395] "U" "U" "U" "U" ...
##  $ famsize   : chr [1:395] "GT3" "GT3" "LE3" "GT3" ...
##  $ Pstatus   : chr [1:395] "A" "T" "T" "T" ...
##  $ Medu      : num [1:395] 4 1 1 4 3 4 2 4 3 3 ...
##  $ Fedu      : num [1:395] 4 1 1 2 3 3 2 4 2 4 ...
##  $ Mjob      : chr [1:395] "at_home" "at_home" "at_home" "health" ...
##  $ Fjob      : chr [1:395] "teacher" "other" "other" "services" ...
##  $ reason    : chr [1:395] "course" "course" "other" "home" ...
##  $ guardian  : chr [1:395] "mother" "father" "mother" "mother" ...
##  $ traveltime: num [1:395] 2 1 1 1 1 1 1 2 1 1 ...
##  $ studytime : num [1:395] 2 2 2 3 2 2 2 2 2 2 ...
##  $ failures  : num [1:395] 0 0 3 0 0 0 0 0 0 0 ...
##  $ schoolsup : chr [1:395] "yes" "no" "yes" "no" ...
##  $ famsup    : chr [1:395] "no" "yes" "no" "yes" ...
##  $ paid      : chr [1:395] "no" "no" "yes" "yes" ...
##  $ activities: chr [1:395] "no" "no" "no" "yes" ...
##  $ nursery   : chr [1:395] "yes" "no" "yes" "yes" ...
##  $ higher    : chr [1:395] "yes" "yes" "yes" "yes" ...
##  $ internet  : chr [1:395] "no" "yes" "yes" "yes" ...
##  $ romantic  : chr [1:395] "no" "no" "no" "yes" ...
##  $ famrel    : num [1:395] 4 5 4 3 4 5 4 4 4 5 ...
```

```
##  $ freetime  : num [1:395] 3 3 3 2 3 4 4 1 2 5 ...
##  $ goout     : num [1:395] 4 3 2 2 2 2 4 4 2 1 ...
##  $ Dalc      : num [1:395] 1 1 2 1 1 1 1 1 1 1 ...
##  $ Walc      : num [1:395] 1 1 3 1 2 2 1 1 1 1 ...
##  $ health    : num [1:395] 3 3 3 5 5 5 3 1 1 5 ...
##  $ absences  : num [1:395] 6 4 10 2 4 10 0 6 0 0 ...
##  $ G1        : num [1:395] 5 5 7 15 6 15 12 6 16 14 ...
##  $ G2        : num [1:395] 6 5 8 14 10 15 12 5 18 15 ...
##  $ G3        : num [1:395] 6 6 10 15 10 15 11 6 19 15 ...
```

```r
head(df)
```

```
## # A tibble: 6 x 33
##   school sex     age address famsize Pstatus  Medu  Fedu Mjob     Fjob    reason
##   <chr>  <chr> <dbl> <chr>   <chr>   <chr>   <dbl> <dbl> <chr>    <chr>   <chr>
## 1 GP     F        18 U       GT3     A           4     4 at_home  teacher course
## 2 GP     F        17 U       GT3     T           1     1 at_home  other   course
## 3 GP     F        15 U       LE3     T           1     1 at_home  other   other
## 4 GP     F        15 U       GT3     T           4     2 health   servic~ home
## 5 GP     F        16 U       GT3     T           3     3 other    other   home
## 6 GP     M        16 U       LE3     T           4     3 services other   reput~
## # i 22 more variables: guardian <chr>, traveltime <dbl>, studytime <dbl>,
## #   failures <dbl>, schoolsup <chr>, famsup <chr>, paid <chr>,
## #   activities <chr>, nursery <chr>, higher <chr>, internet <chr>,
## #   romantic <chr>, famrel <dbl>, freetime <dbl>, goout <dbl>, Dalc <dbl>,
## #   Walc <dbl>, health <dbl>, absences <dbl>, G1 <dbl>, G2 <dbl>, G3 <dbl>
```

```r
dim(df)
```

```
## [1] 395  33
```

```r
names(df)
```

```
##  [1] "school"     "sex"        "age"        "address"    "famsize"
##  [6] "Pstatus"    "Medu"       "Fedu"       "Mjob"       "Fjob"
## [11] "reason"     "guardian"   "traveltime" "studytime"  "failures"
## [16] "schoolsup"  "famsup"     "paid"       "activities" "nursery"
## [21] "higher"     "internet"   "romantic"   "famrel"     "freetime"
## [26] "goout"      "Dalc"       "Walc"       "health"     "absences"
## [31] "G1"         "G2"         "G3"
```

We can see that our dataset 'df' has 33 different variables, or columns, 32 of them will be our predictors. We can also see that 'df' has 395 rows, or observations.

A quick description of every variable can be found in the variable information section in the UCI Machine Learning Repository (Again, for more information on sources and websites, go to the References pages). We provide the definitions given by the info there in this report here:

school: Student's school, 'GP' or 'MS'

sex: Male or female

age: 15 to 22

address: Urban or rural

famsize: Family size, <= 3 or > 3

Pstatus: Parents living together or apart

Medu & Fedu: Mother and Father's Education level from 0 to 4

0 - None 1 - Primary 2 - Up to 9th grade 3 - Secondary 4 - Higher education

Mjob & Fjob: Mother and Father's job

- Teacher
- Health
- Services
- At Home
- Other

reason: Reason to choose this school

- Close to home
- Reputation
- Course
- Other

guardian: Student's guardian

- Mother
- Father
- Other

traveltime: Home to school travel time

studytime: Weekly study time

failures: Number of past class failures

schoolsup: Extra educational support

famsup: Family educational support

paid: Extra paid classes within the course

activities: Involved in extracurricular activities

nursery: Attended nursery school

higher: Wants to take higher education

internet: Internet access at home

romantic: In a romantic relationship

famrel: Quality of family relationships

freetime: Free time after school

goout: Going out with friends

Dalc: workday alcohol consumption

Walc: weekend alcohol consumption

health: current health status

absences: Number of school absences

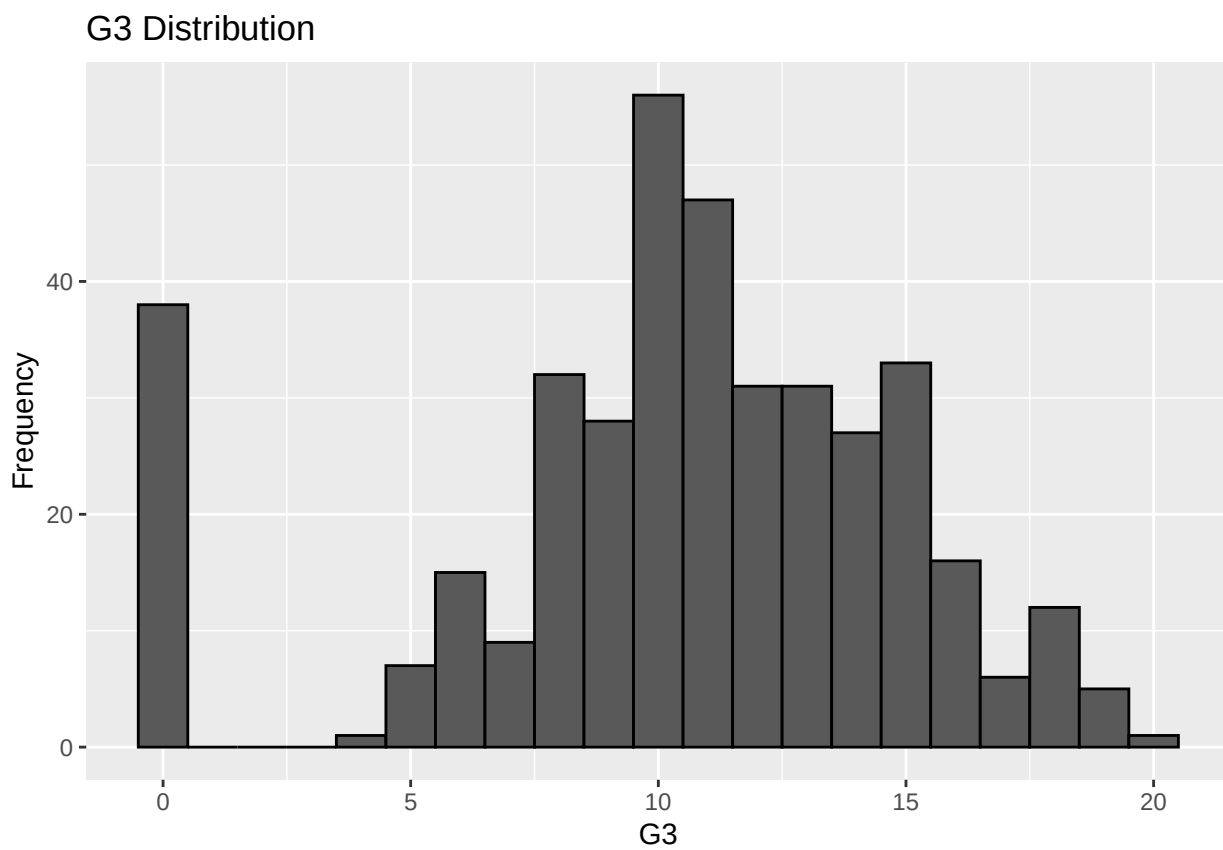
G1: First period grade

G2: Second Period Grade

G3: Final (3rd period) Grade

We will move on by seeing how the different G1, G2, and G3, scores are distributed and attempt to make observations and analyses based on that data.

```
## Start off by seeing how G3 is distributed
df %>%
  ggplot(aes(G3)) +
  geom_histogram(binwidth = 1, color = "black") +
  ggtitle("G3 Distribution") +
  xlab("G3") +
  ylab("Frequency")
```



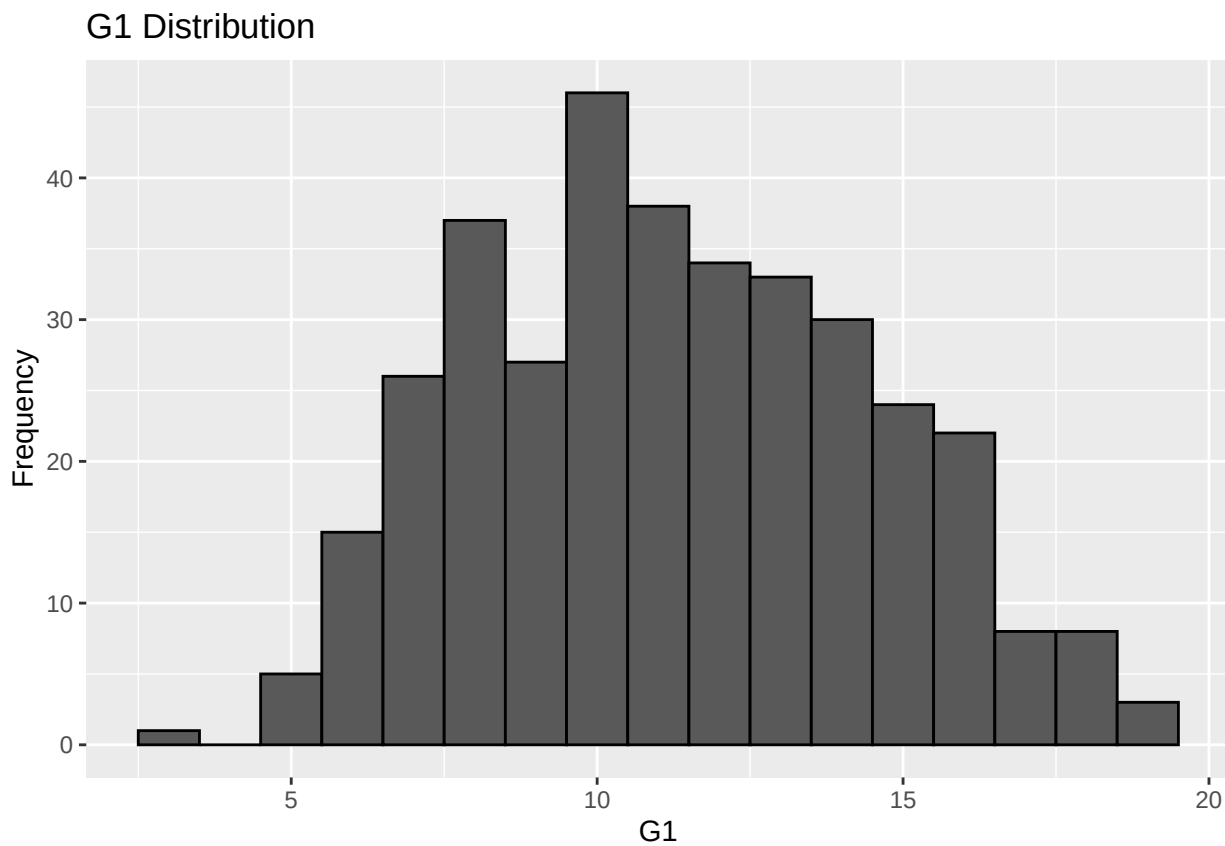

```
## Approximately normally distributed but large outlier at 0.
```

We can see that the values of G3 are approximately normally distributed, but there is a large outlier at 0. This would only be possible if a student didn't take the test, or got every single question wrong. Regardless, this will not help in our prediction, so we will get rid of the outliers with the following code:

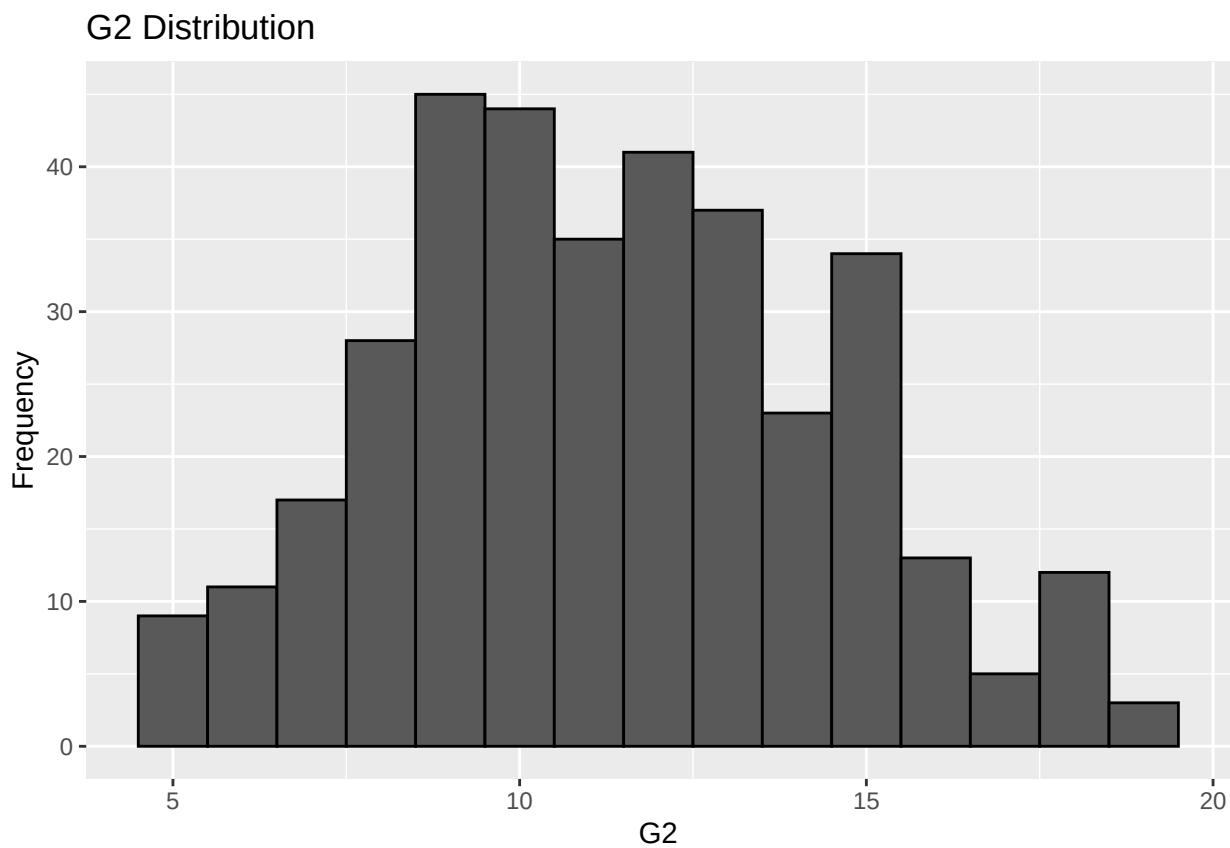
```r
## Getting rid of the outlier at 0
df_new <- df %>%
  filter(G3 != 0)
```

Our filtered data set without the outliers will be named 'df_new'. Next we will view the distributions of G1 and G2.

```r
## We will view the distributions for G1 and G2
df_new %>%
  ggplot(aes(G1)) +
  geom_histogram(binwidth = 1, color = "black") +
  ggtitle("G1 Distribution") +
  xlab("G1") +
  ylab("Frequency")
```




```r
df_new %>%
  ggplot(aes(G2)) +
  geom_histogram(binwidth = 1, color = "black") +
  ggtitle("G2 Distribution") +
  xlab("G2") +
  ylab("Frequency")
```




```
## Both are approximately normally distributed
```

We can see that both G1 and G2 are also approximately distributed. We will use the following code to check for outliers:

```
## Check for 0's
sum(df_new$G1 == 0)
```

```
## [1] 0
```

```
sum(df_new$G2 == 0)
```

```
## [1] 0
```

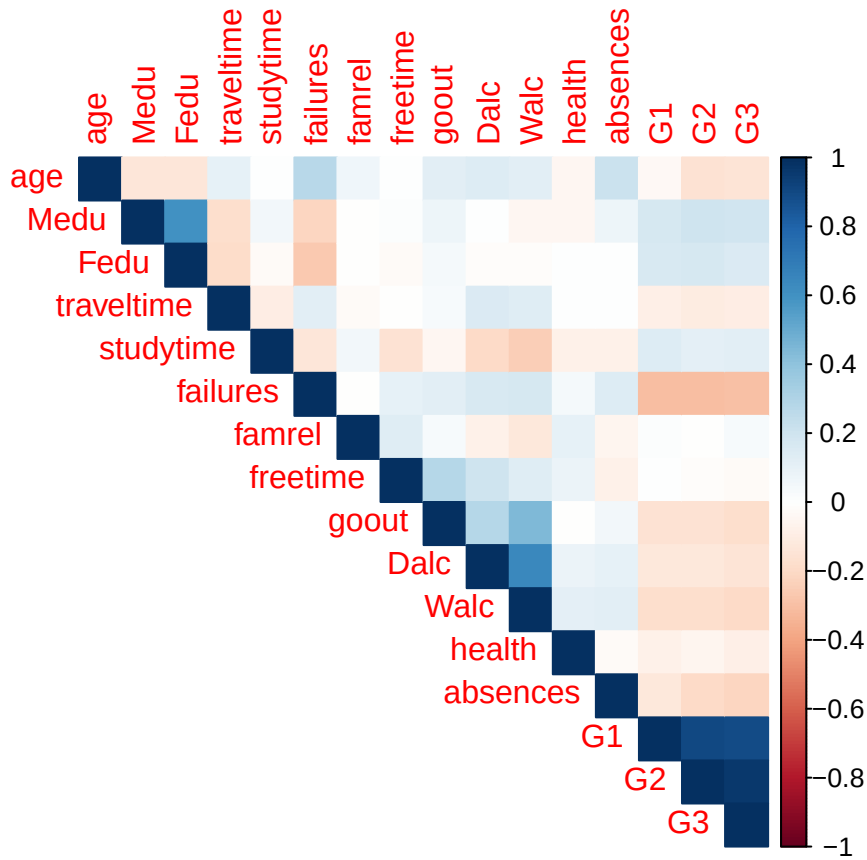
```
## There are no zeros or outliers
```

We see that there are no 0's or outliers in G1 and G2, so we don't need to clean those. We are now done cleaning our dataset since we have gotten rid of all the outliers and NA values.

Data Analysis

We will first create a correlation chart with the numeric predictors to see the correlation between predictors and the correlation to G3, our target variable. Seeing the correlation of varibales will help us to make more accurate predictions by using only the ones that clearly have an effect on G3.

```r
## Checking whether predictors are independent or not
numeric_grade <- df_new[, sapply(df_new, is.numeric)]
correlation_matrix <- cor(numeric_grade)
corrplot(correlation_matrix, method = "color", type = "upper", tl.cex = 1)
```




```r
## Medu and Fedu highly correlated
## Dalc and Walc are highly correlated
## G1 and G2 are also highly correlated
```

From this correlation chart we can make a few observations. G1 and G2 are highly correlated with G3. This makes sense because G1 and G2 are the first two period grades and G3 is the final period grade.

We see that weekday alcohol is highly associated with weekend alcohol. Again, no big surprise, because if someone tends to drink a lot on weekdays then they would most likely do the same on weekends, and vice versa.

We can also observe that the mother and father's education level typically are similar and highly correlated.

However, other than G1 and G2, the majority of numeric predictors and G3 have a correlation value somewhere within the range of $-0.5 <= corr <= 0.5$. This means that there isn't that strong of a correlation between the majority of the numeric predictors and the final G3 score. Using these in our prediction could potentially cause more random noise than actual results.

We will continue to observe other non-numeric predictors to try to see any type of correlation or pattern, but first we will split up the dataset 'df_new' into a validation set and a final test set, which we will not test on until we want our final RMSE value. Our validation set will be called 'grade' and our final test set will

be called 'final_test'. We will split the data into a 9:1 ratio. A 9:1 ratio will allow us to reasonably predict our target using the validation set without overfitting by making the validation set too small.

```r
## Splitting into our training set and final test set
set.seed(1)
train_index <- createDataPartition(df_new$G3, p = 0.9, list = FALSE)
grade <- df_new[train_index,]
final_test <- df_new[-train_index,]
```

Let's get familiar with the new dataset:

```r
## Getting familiar with the new dataset
str(grade)
```

```
## tibble [323 x 33] (S3: tbl_df/tbl/data.frame)
##  $ school    : chr [1:323] "GP" "GP" "GP" "GP" ...
##  $ sex       : chr [1:323] "F" "F" "F" "F" ...
##  $ age       : num [1:323] 18 17 15 15 16 16 16 17 15 15 ...
##  $ address   : chr [1:323] "U" "U" "U" "U" ...
##  $ famsize   : chr [1:323] "GT3" "GT3" "LE3" "GT3" ...
##  $ Pstatus   : chr [1:323] "A" "T" "T" "T" ...
##  $ Medu      : num [1:323] 4 1 1 4 3 4 2 4 3 3 ...
##  $ Fedu      : num [1:323] 4 1 1 2 3 3 2 4 2 4 ...
##  $ Mjob      : chr [1:323] "at_home" "at_home" "at_home" "health" ...
##  $ Fjob      : chr [1:323] "teacher" "other" "other" "services" ...
##  $ reason    : chr [1:323] "course" "course" "other" "home" ...
##  $ guardian  : chr [1:323] "mother" "father" "mother" "mother" ...
##  $ traveltime: num [1:323] 2 1 1 1 1 1 1 2 1 1 ...
##  $ studytime : num [1:323] 2 2 2 3 2 2 2 2 2 2 ...
##  $ failures  : num [1:323] 0 0 3 0 0 0 0 0 0 0 ...
##  $ schoolsup : chr [1:323] "yes" "no" "yes" "no" ...
##  $ famsup    : chr [1:323] "no" "yes" "no" "yes" ...
##  $ paid      : chr [1:323] "no" "no" "yes" "yes" ...
##  $ activities: chr [1:323] "no" "no" "no" "yes" ...
##  $ nursery   : chr [1:323] "yes" "no" "yes" "yes" ...
##  $ higher    : chr [1:323] "yes" "yes" "yes" "yes" ...
##  $ internet  : chr [1:323] "no" "yes" "yes" "yes" ...
##  $ romantic  : chr [1:323] "no" "no" "no" "yes" ...
##  $ famrel    : num [1:323] 4 5 4 3 4 5 4 4 4 5 ...
##  $ freetime  : num [1:323] 3 3 3 2 3 4 4 1 2 5 ...
##  $ goout     : num [1:323] 4 3 2 2 2 2 4 4 2 1 ...
##  $ Dalc      : num [1:323] 1 1 2 1 1 1 1 1 1 1 ...
##  $ Walc      : num [1:323] 1 1 3 1 2 2 1 1 1 1 ...
##  $ health    : num [1:323] 3 3 3 5 5 5 3 1 1 5 ...
##  $ absences  : num [1:323] 6 4 10 2 4 10 0 6 0 0 ...
##  $ G1        : num [1:323] 5 5 7 15 6 15 12 6 16 14 ...
##  $ G2        : num [1:323] 6 5 8 14 10 15 12 5 18 15 ...
##  $ G3        : num [1:323] 6 6 10 15 10 15 11 6 19 15 ...
```

```r
head(grade)
```

```
## # A tibble: 6 x 33
```

```
##   school sex     age address famsize Pstatus  Medu  Fedu Mjob     Fjob    reason
##   <chr>  <chr> <dbl> <chr>   <chr>   <chr>   <dbl> <dbl> <chr>    <chr>   <chr>
## 1 GP     F        18 U       GT3     A           4     4 at_home  teacher course
## 2 GP     F        17 U       GT3     T           1     1 at_home  other   course
## 3 GP     F        15 U       LE3     T           1     1 at_home  other   other
## 4 GP     F        15 U       GT3     T           4     2 health   servic~ home
## 5 GP     F        16 U       GT3     T           3     3 other    other   home
## 6 GP     M        16 U       LE3     T           4     3 services other   reput~
## # i 22 more variables: guardian <chr>, traveltime <dbl>, studytime <dbl>,
## #   failures <dbl>, schoolsup <chr>, famsup <chr>, paid <chr>,
## #   activities <chr>, nursery <chr>, higher <chr>, internet <chr>,
## #   romantic <chr>, famrel <dbl>, freetime <dbl>, goout <dbl>, Dalc <dbl>,
## #   Walc <dbl>, health <dbl>, absences <dbl>, G1 <dbl>, G2 <dbl>, G3 <dbl>
```

```r
dim(grade)
```

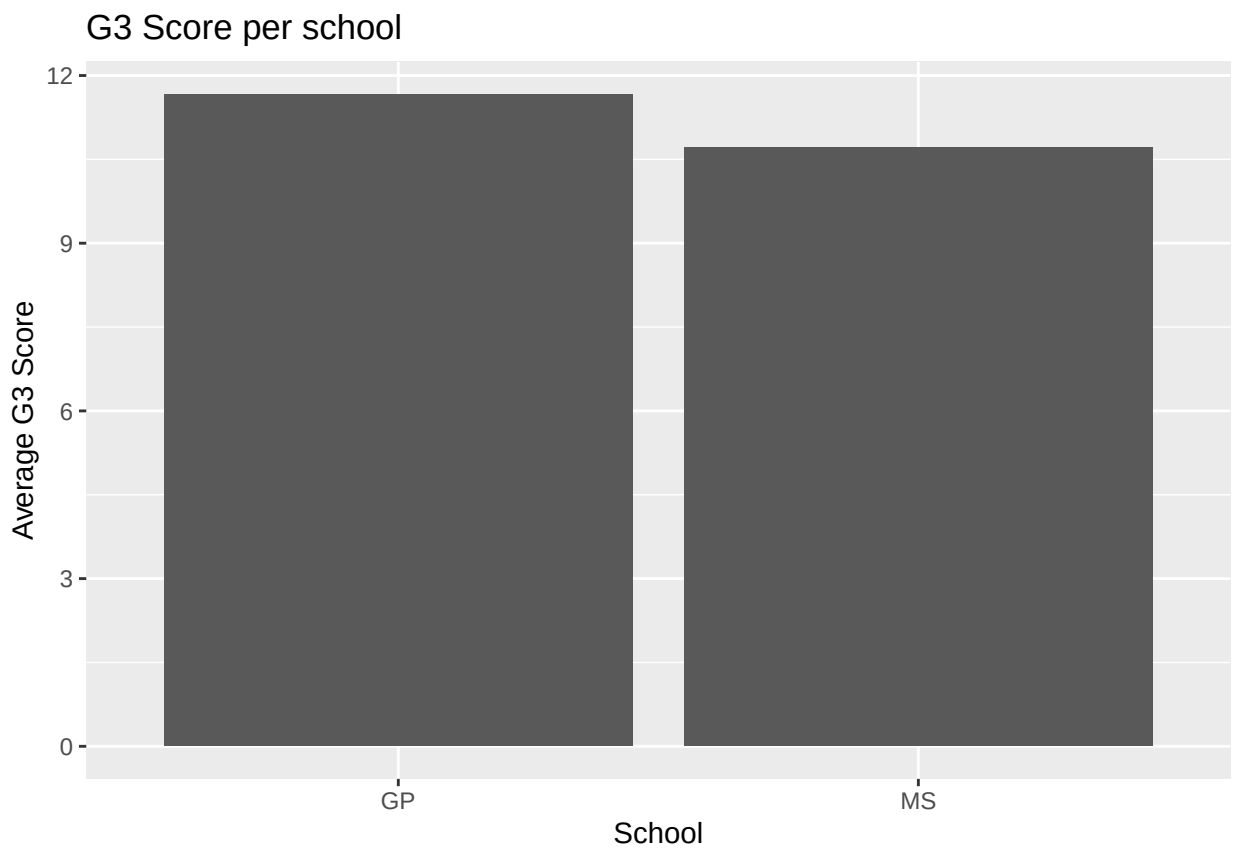
```
## [1] 323  33
```

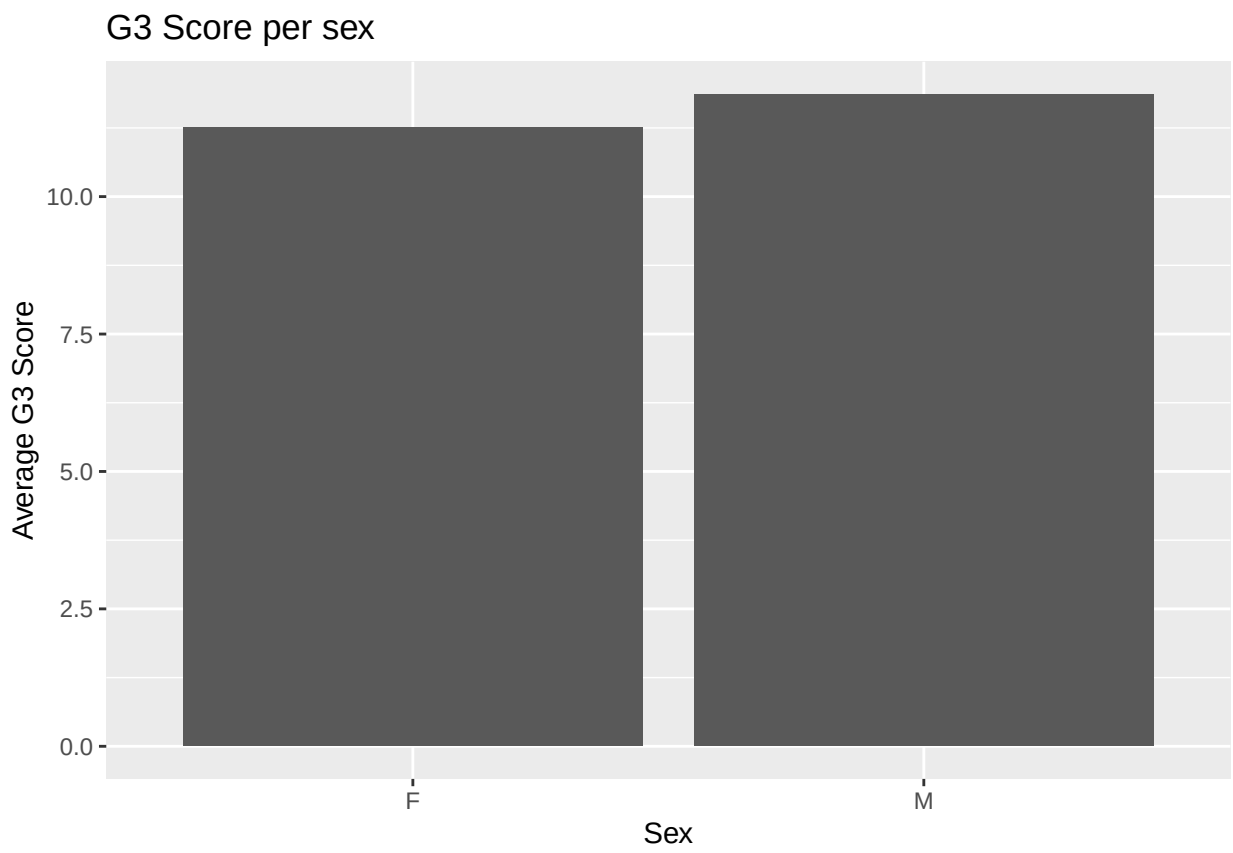
```r
names(grade)
```

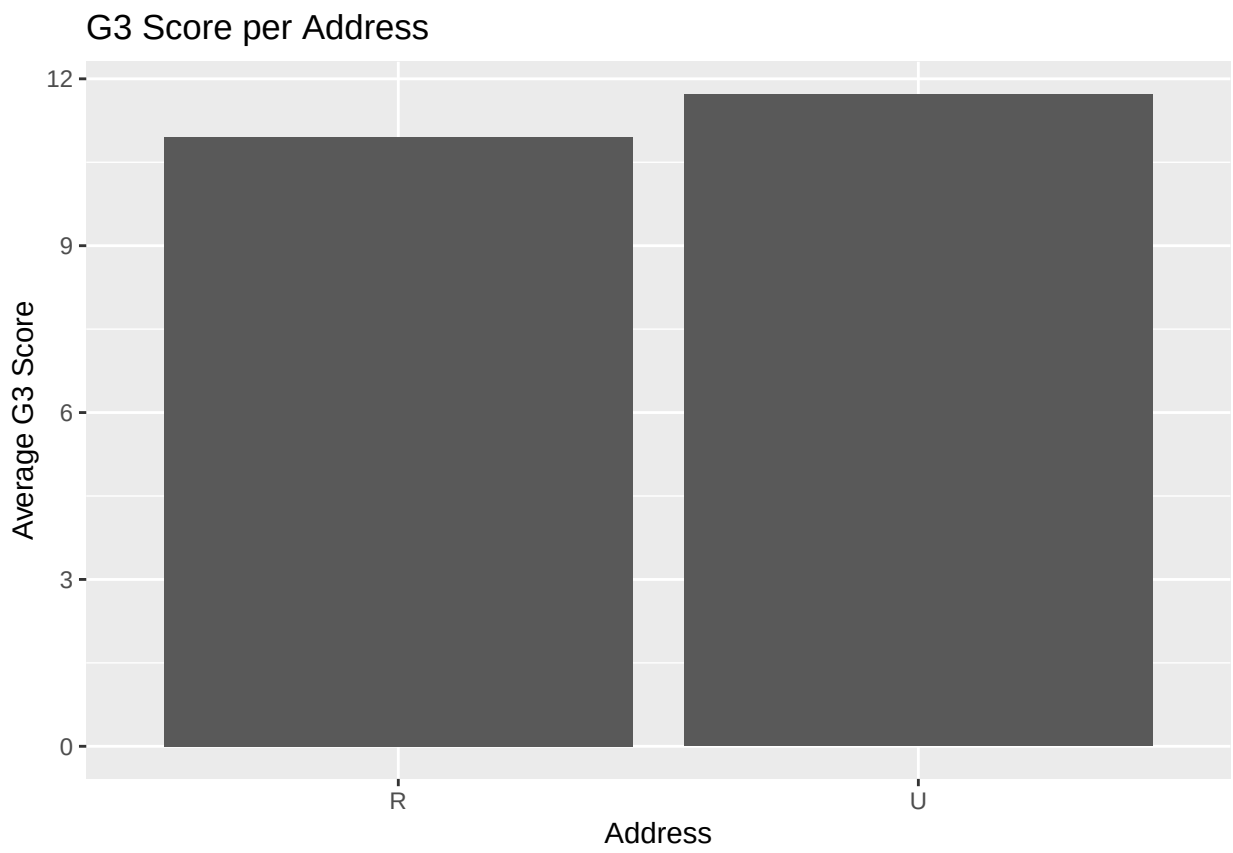
```
##  [1] "school"     "sex"        "age"        "address"    "famsize"
##  [6] "Pstatus"    "Medu"       "Fedu"       "Mjob"       "Fjob"
## [11] "reason"     "guardian"   "traveltime" "studytime"  "failures"
## [16] "schoolsup"  "famsup"     "paid"       "activities" "nursery"
## [21] "higher"     "internet"   "romantic"   "famrel"     "freetime"
## [26] "goout"      "Dalc"       "Walc"       "health"     "absences"
## [31] "G1"         "G2"         "G3"
```

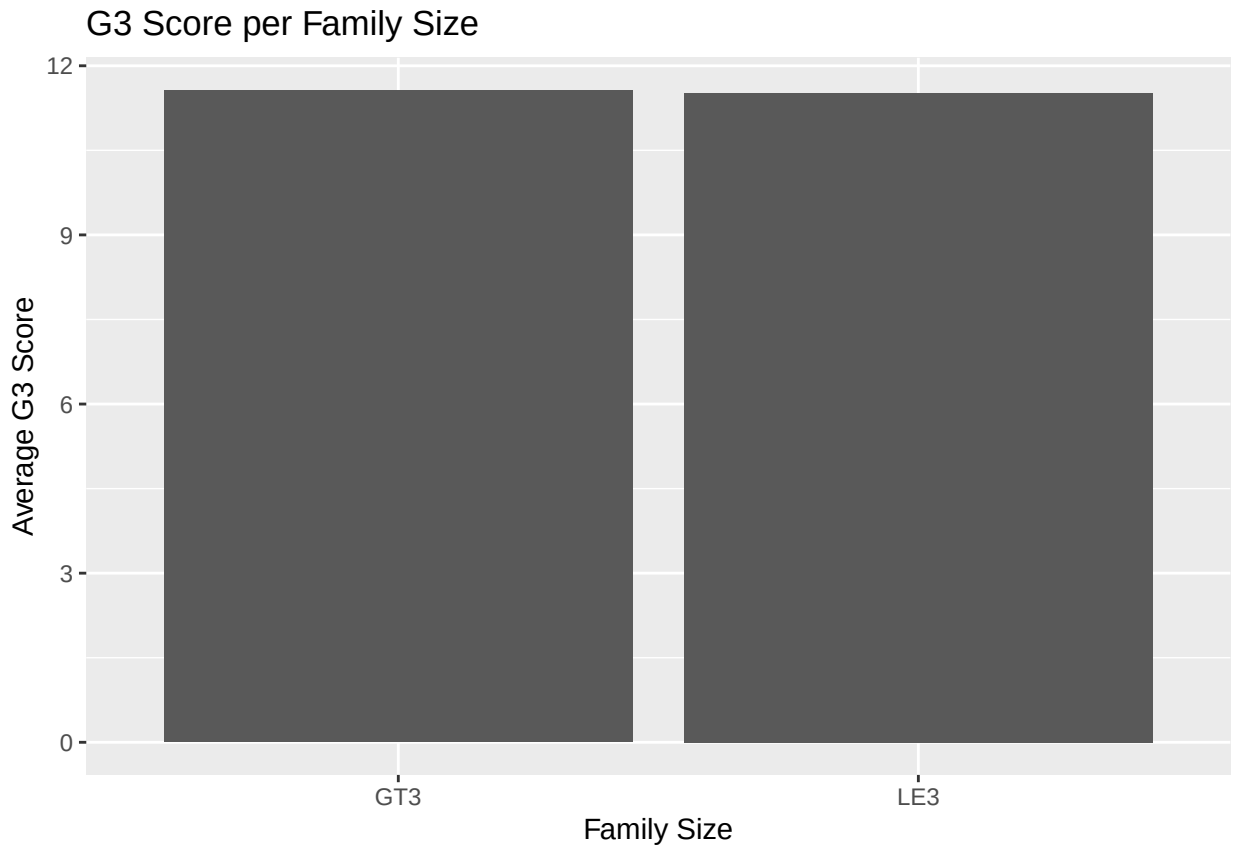
We can see that our predictors and column numbers remain the same, but now we only have 323 observations, because 10 percent of the observations were partitioned to 'final_test'.

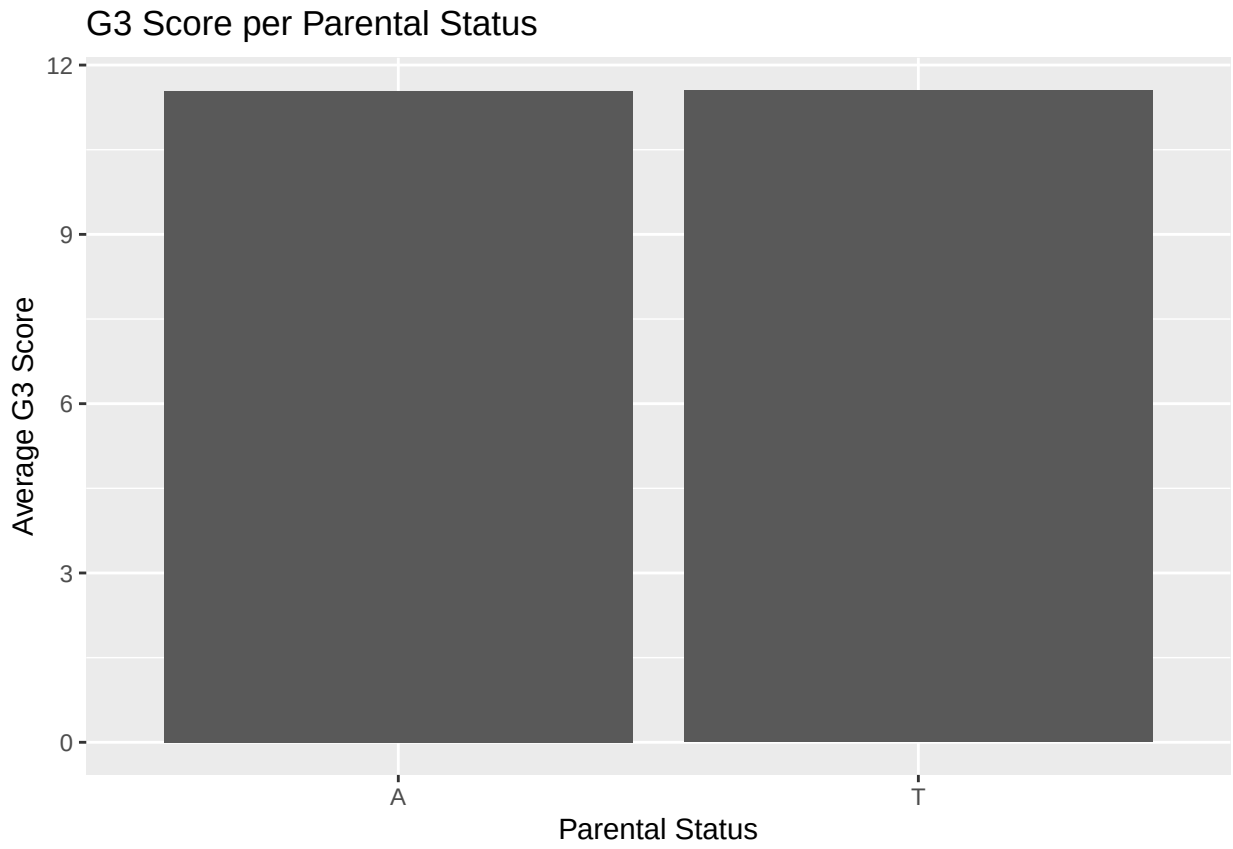
Other than numeric values and integers, another large category of variable types is binary values. We will create several bar plots to observe the relations between binary values and G3 scores.

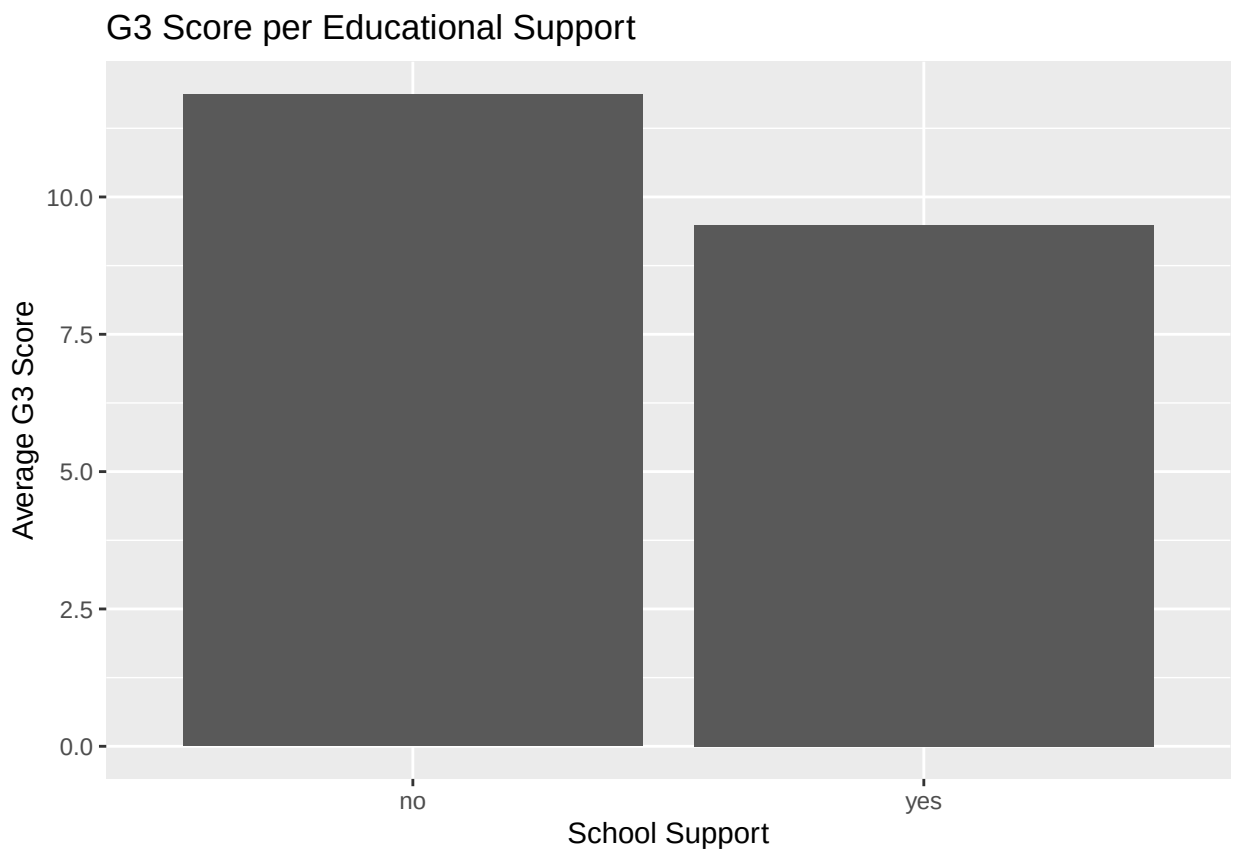

G3 Score per school
12
9
6
3
0
Average G3 Score
GP
MS
School

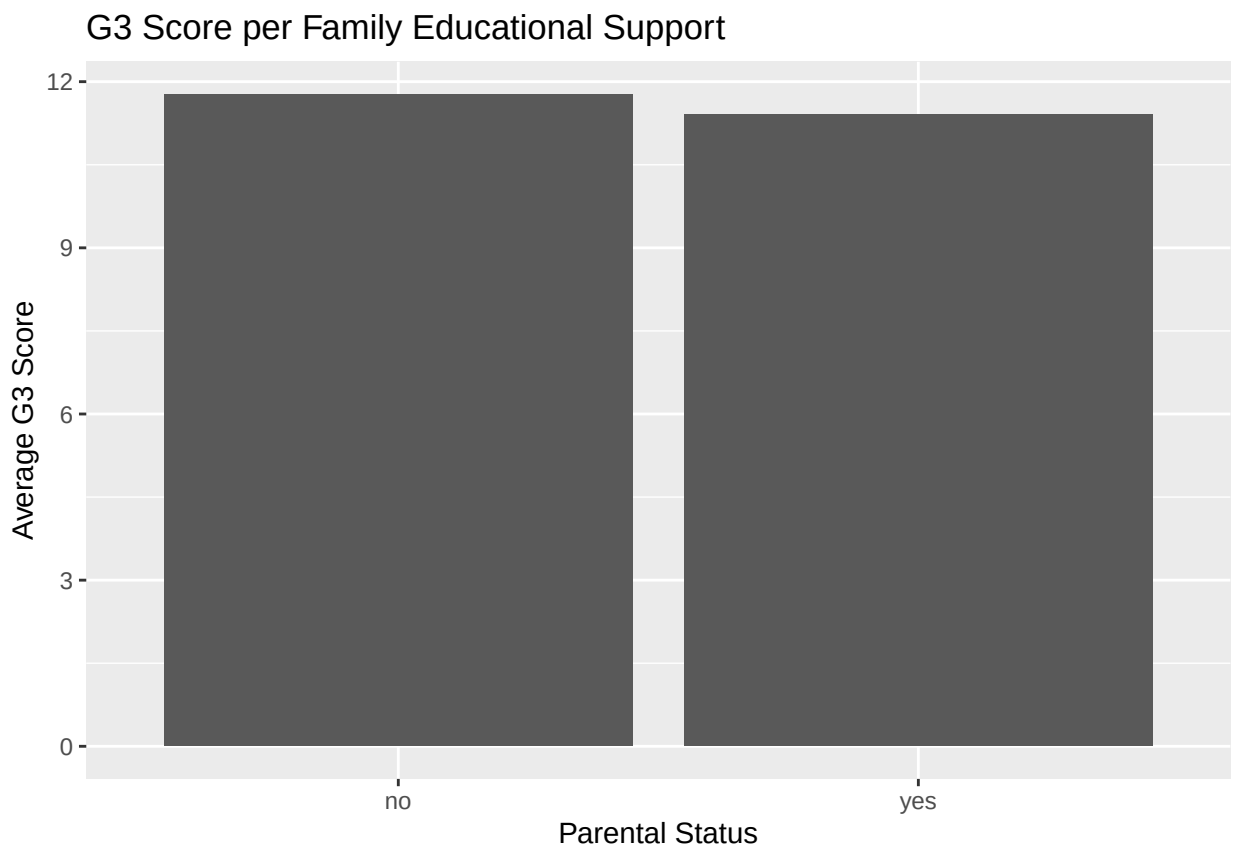

G3 Score per sex
Average G3 Score
10.0
7.5
5.0
2.5
0.0
F
M
Sex

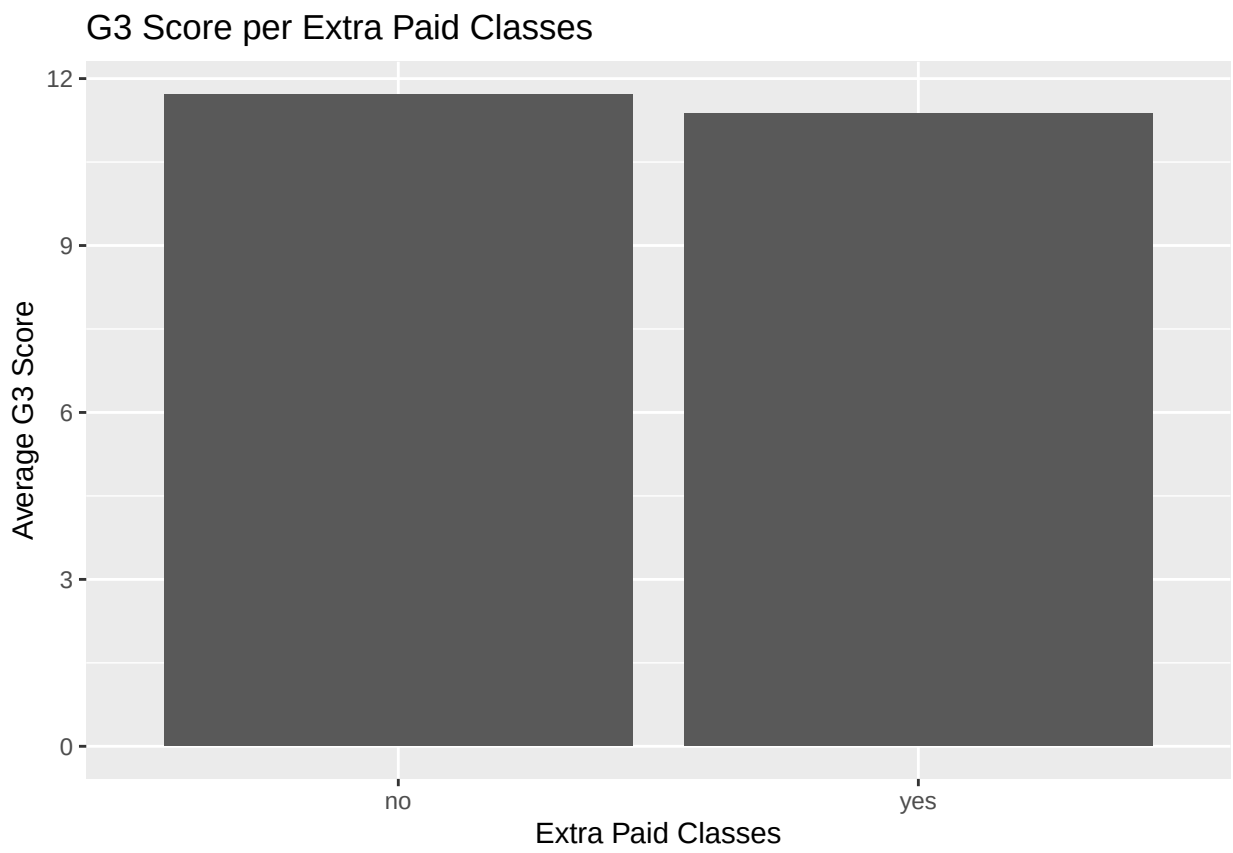

G3 Score per Address
Average G3 Score
12
9
6
3
0
R
U
Address

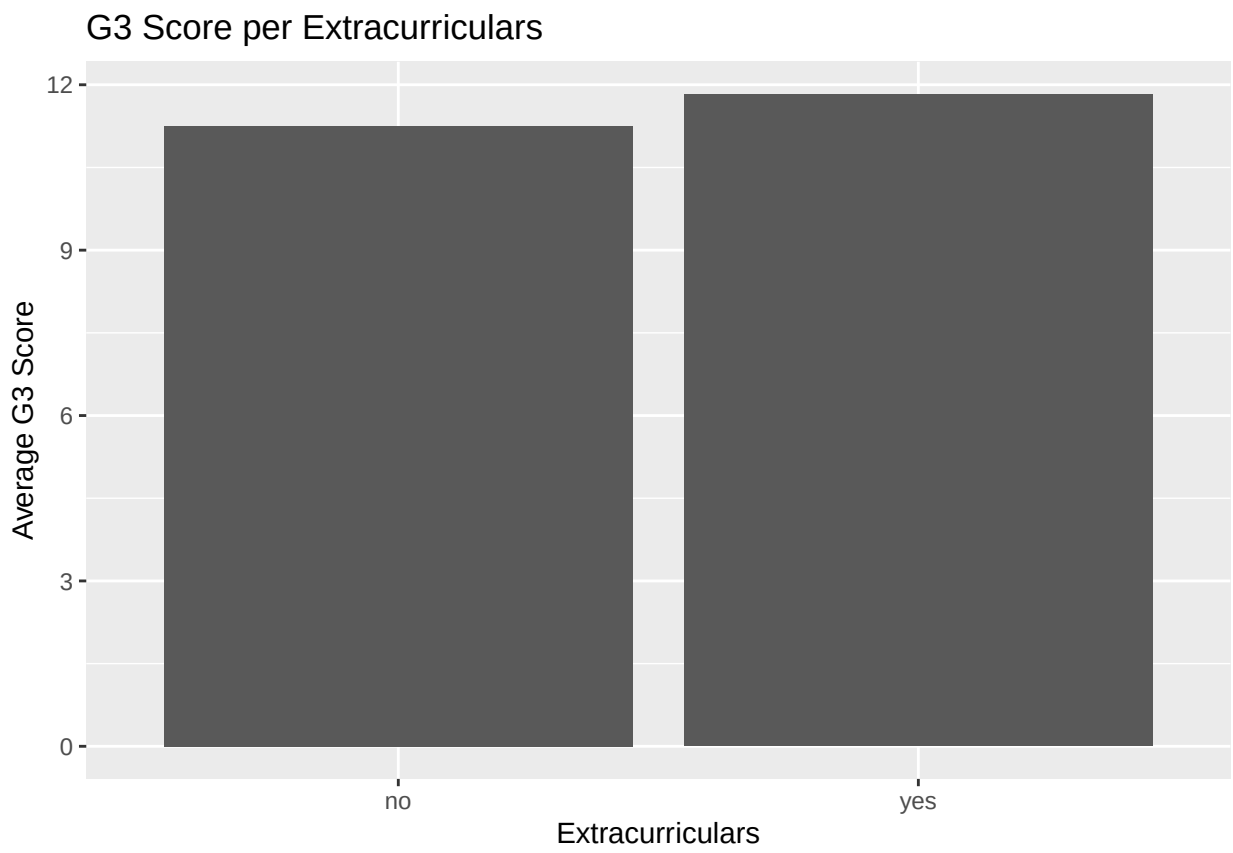

G3 Score per Family Size
12
9
6
3
0
Average G3 Score
GT3
LE3
Family Size

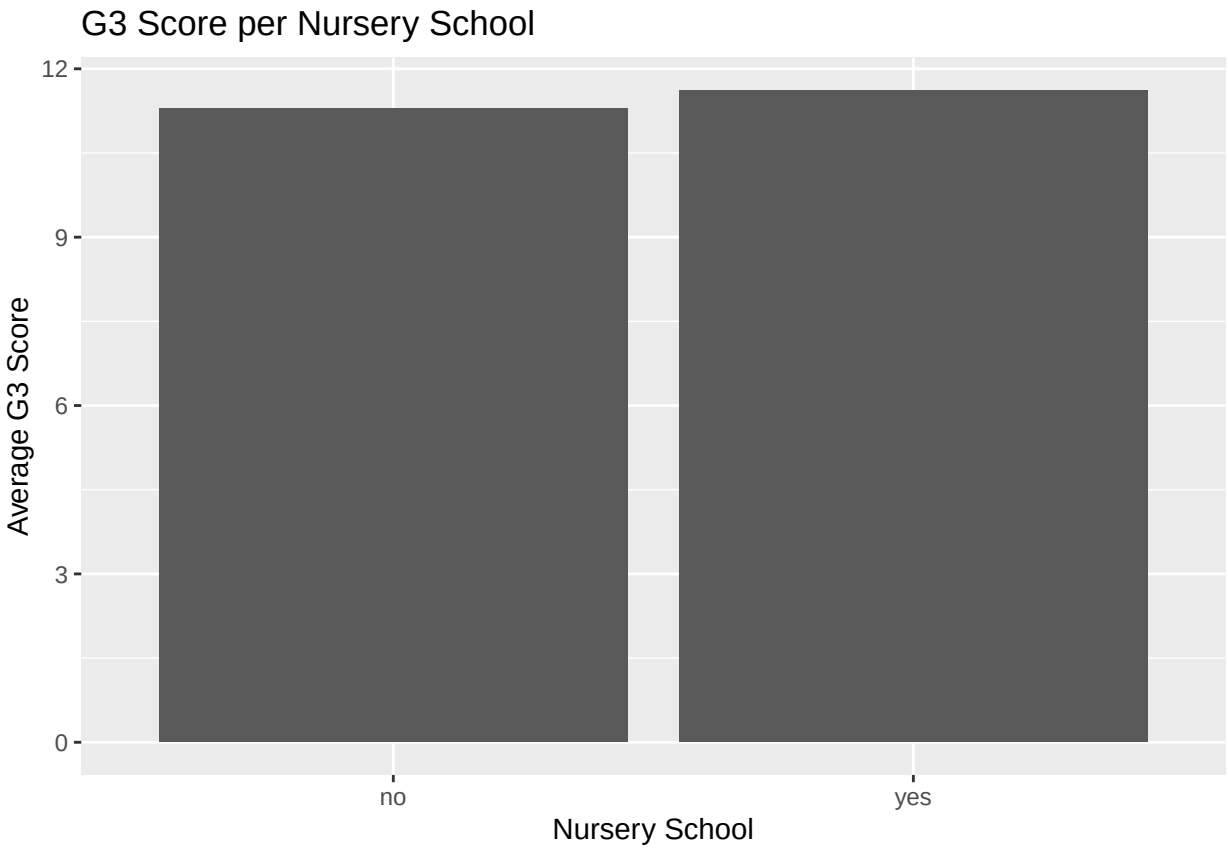

G3 Score per Parental Status
Average G3 Score
12
9
6
3
0
A
T
Parental Status

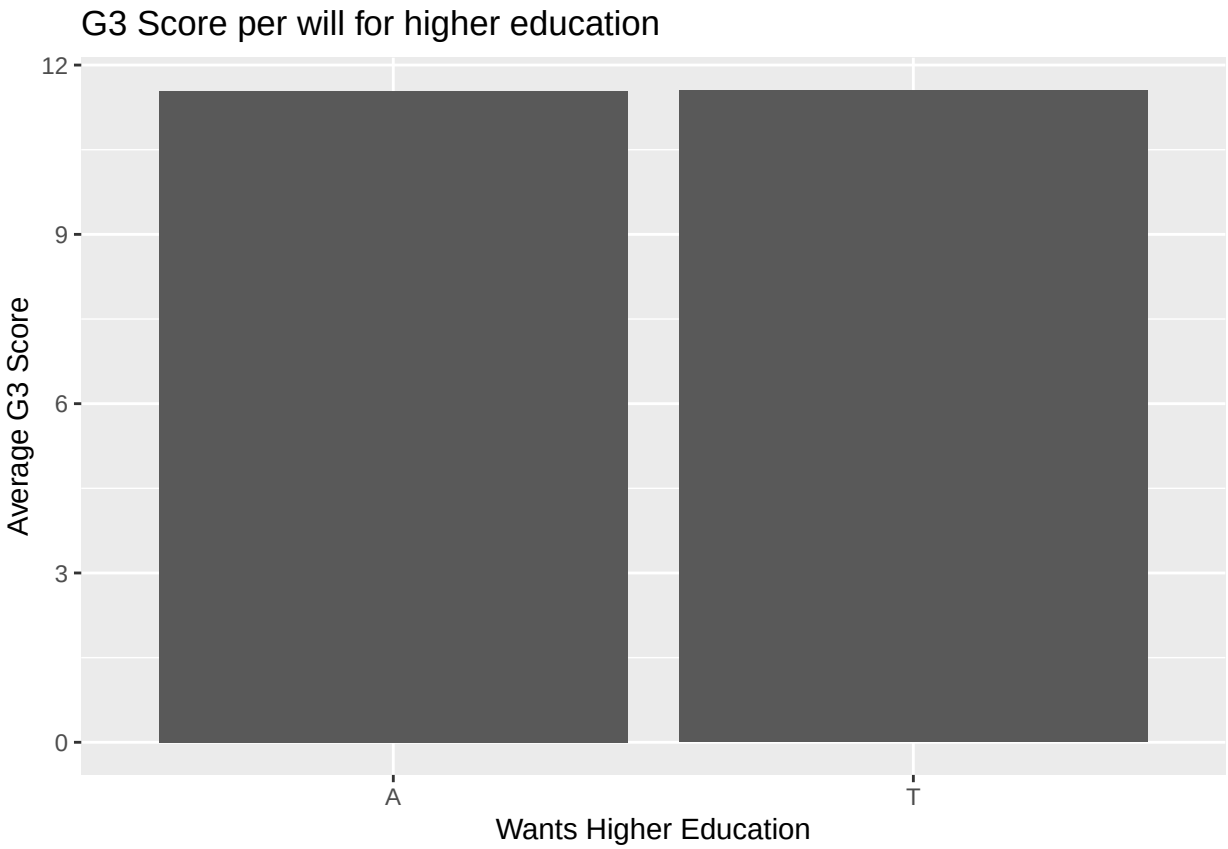

G3 Score per Educational Support
Average G3 Score
10.0
7.5
5.0
2.5
0.0
no
yes
School Support

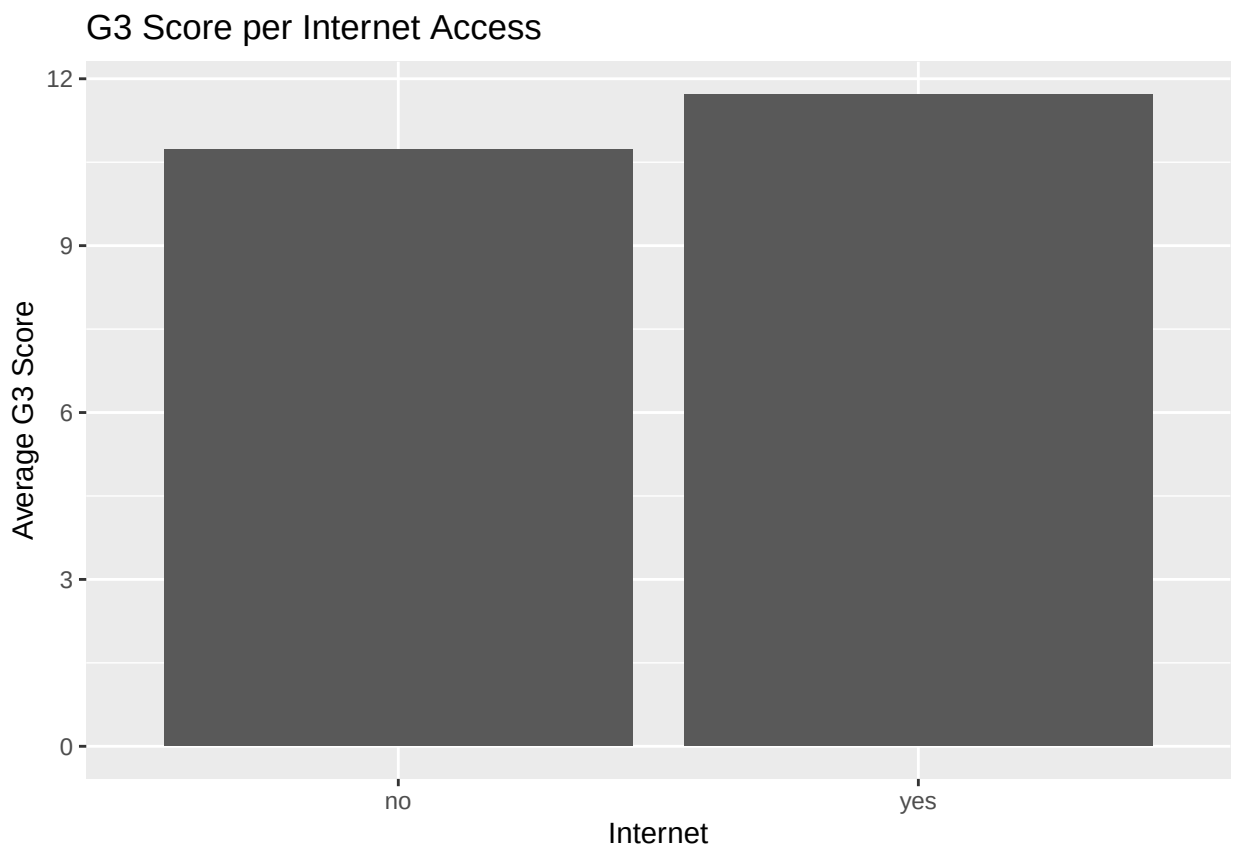

G3 Score per Family Educational Support
Average G3 Score
12
9
6
3
0
no
yes
Parental Status

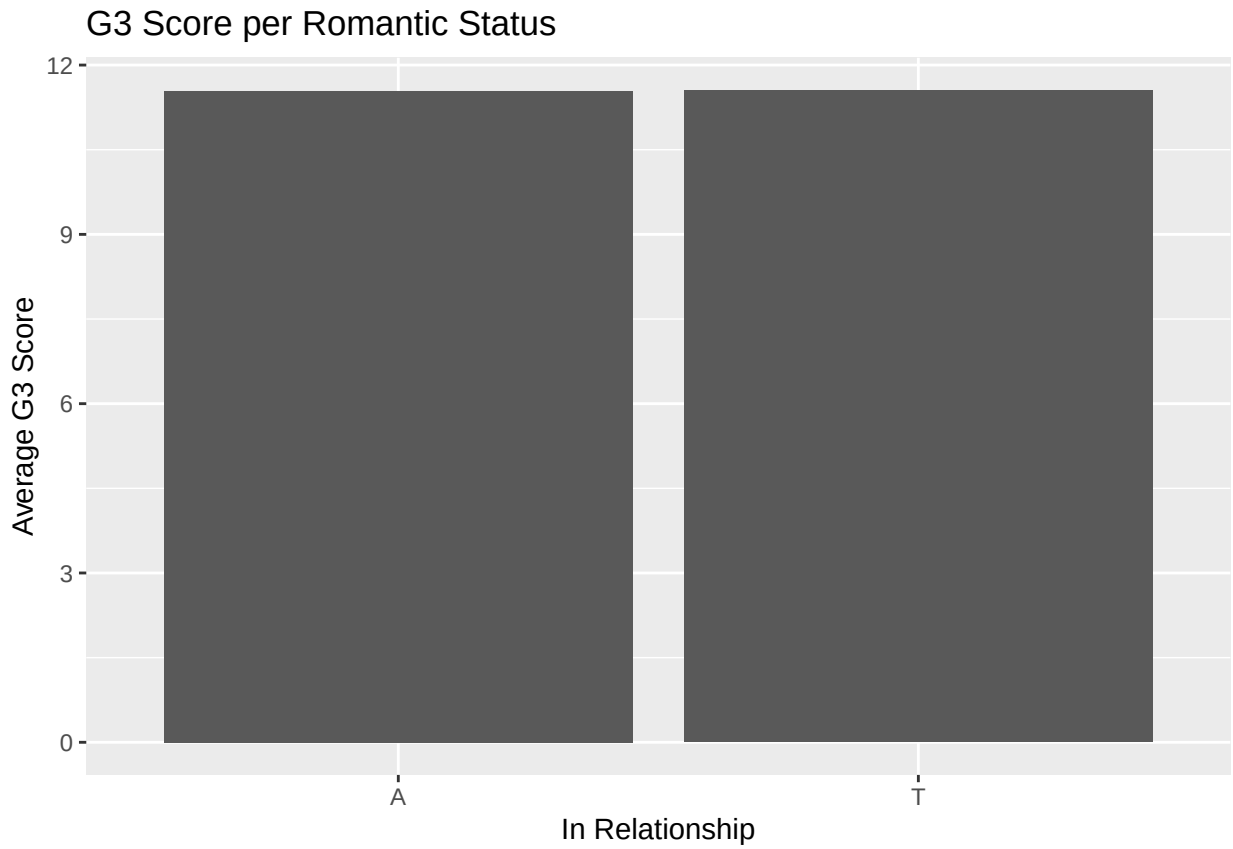

G3 Score per Extra Paid Classes
Average G3 Score
12
9
6
3
0
no
yes
Extra Paid Classes


G3 Score per Extracurriculars
Average G3 Score
12
9
6
3
0
no
yes
Extracurriculars


G3 Score per Nursery School
Average G3 Score
12
9
6
3
0
no
yes
Nursery School


G3 Score per will for higher education
Average G3 Score
12
9
6
3
0
A
T
Wants Higher Education

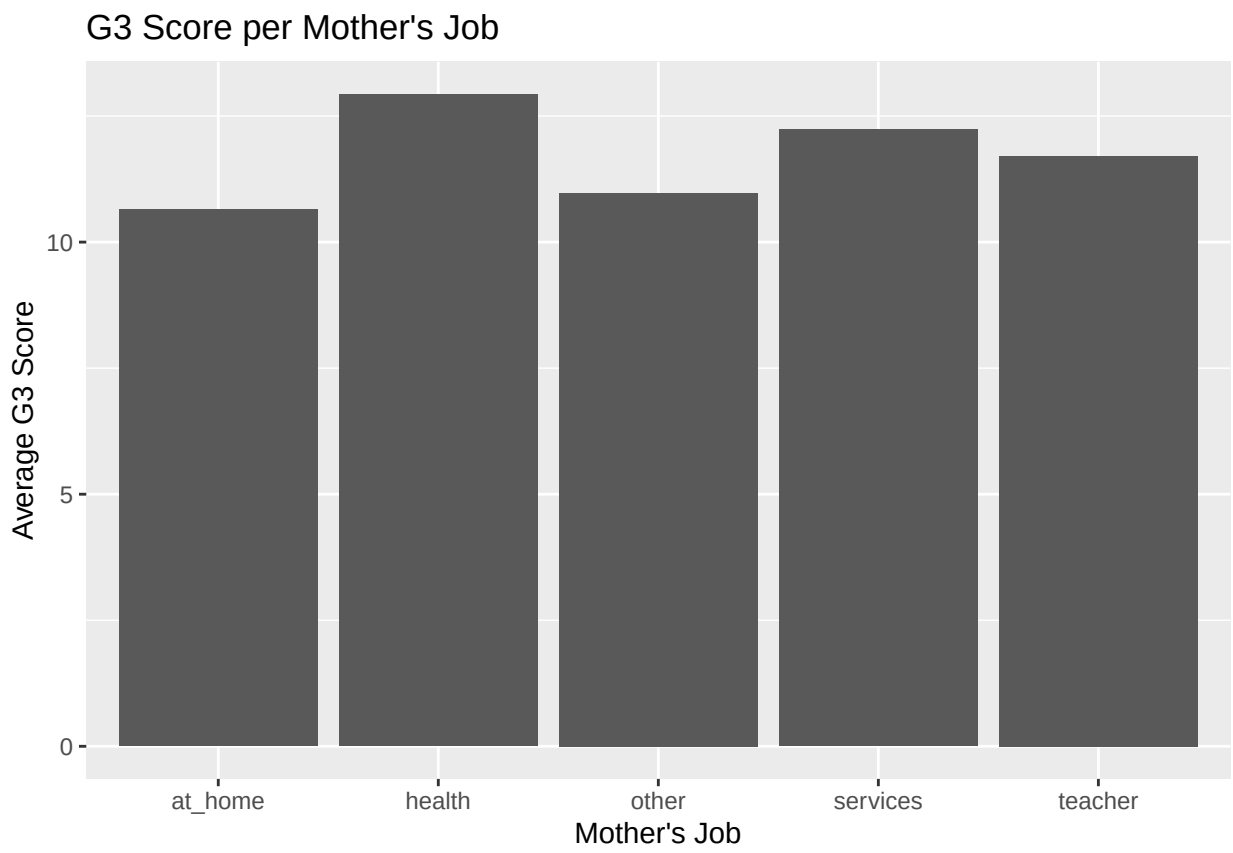

G3 Score per Internet Access
Average G3 Score
12
9
6
3
0
no
yes
Internet

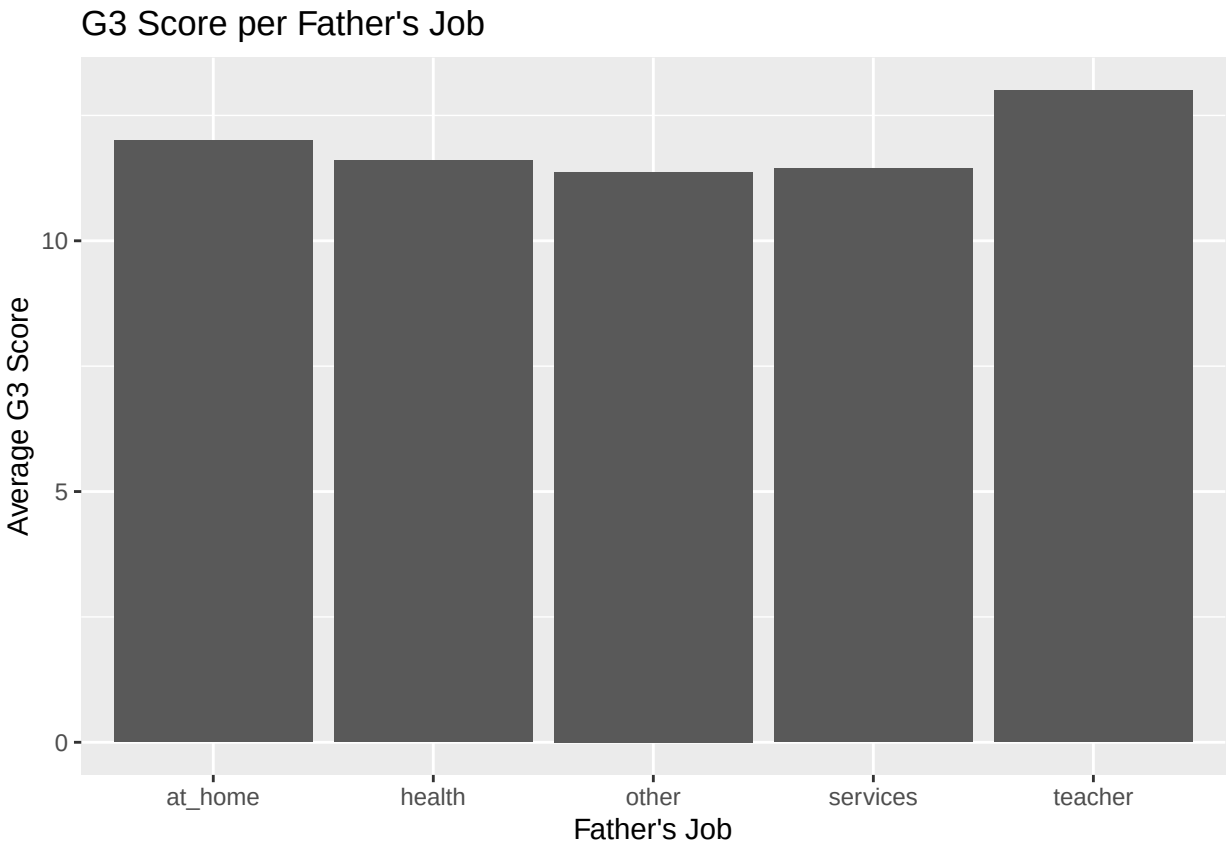


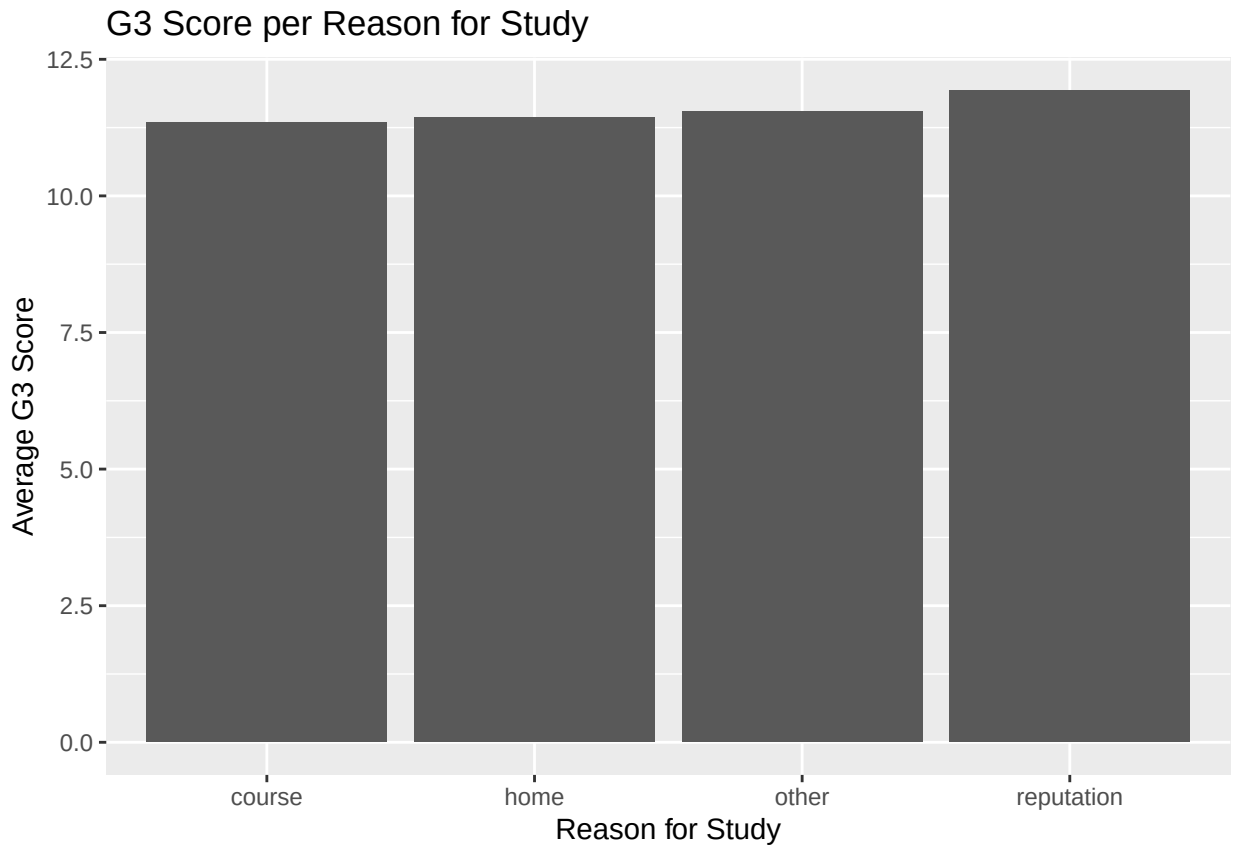
Observing the several bar plots of binary variables and G3 scores here, we can see that the majority of them do not show a drastic difference or effect on G3 scores. Any differences are generally very minimal. Attempting to use these binary predictors in our model could, like many of the numeric predictors, create random noise that will screw up the accuracy of the model. If we use them we should be wary of the consequences.

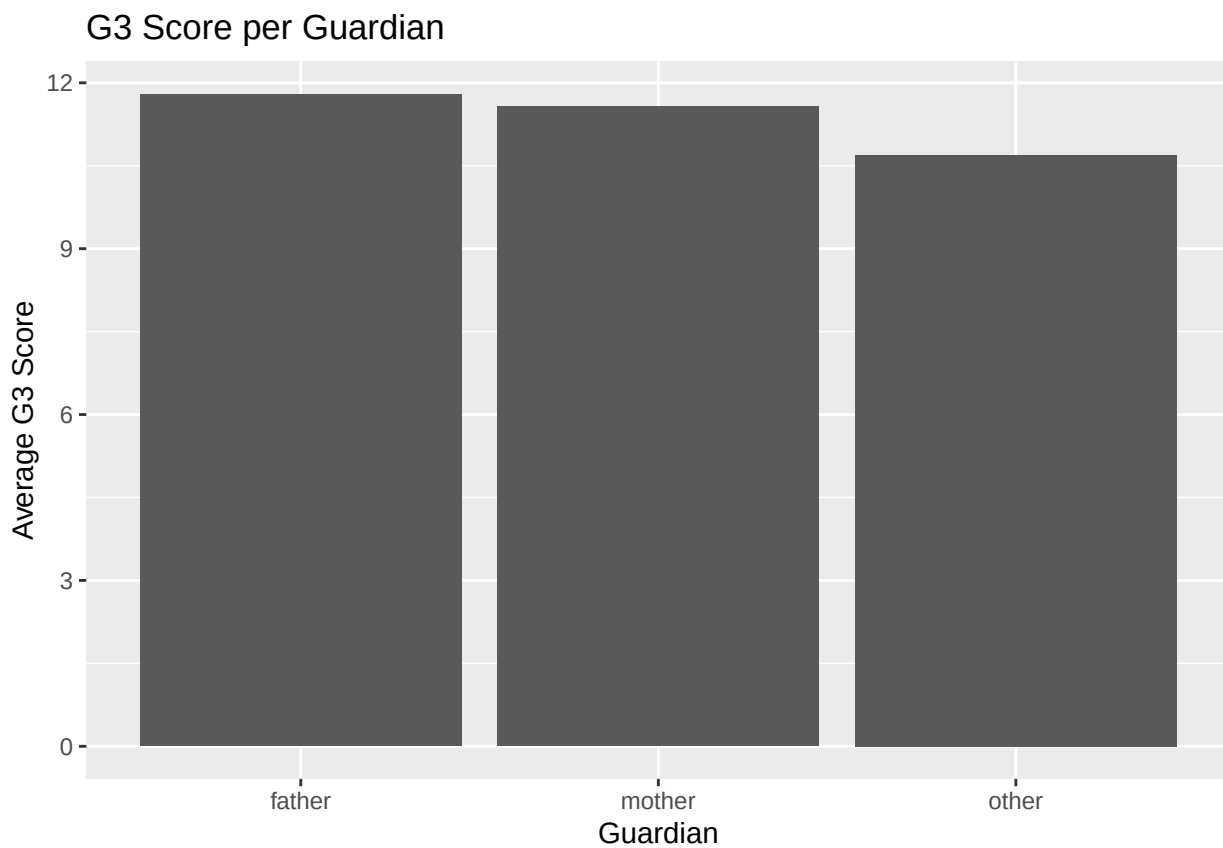
After having observed the differences between binary variables (or lack thereof), we will continue by observing bar plots of categorical variables.

The following code will give us the bar plots:


G3 Score per Mother's Job
Average G3 Score
10
5
0
at_home
health
other
services
teacher
Mother's Job


G3 Score per Father's Job
Average G3 Score
10
5
0
at_home
health
other
services
teacher
Father's Job


G3 Score per Reason for Study
12.5
10.0
7.5
5.0
2.5
0.0
Average G3 Score
course
home
other
reputation
Reason for Study



From the bar plots we can see that the values of G3 scores aren't as identical as with the binary predictors. However, the differences are still very minimal and can't be used effectively within our prediction without potentially causing random noise.

Linear Model

After assessing the data from the plots of the different predictors, we've deduced that the only significantly effective predictors for predicting the target, G3, are G1 and G2. Using other predictors that did not show that much effect on the G3 scores could have the potential to cause random noise. Our first model will be a baseline linear model, with added biases from the predictors and regularization. G1, G2, and G3 were normally distributed so they should work well with a linear model.

We will begin by partitioning 'grade' into a 'training_set' and 'test_set'. We will partition these into an 8:2 ratio. Again, this is because we don't want the test set to be too large, or the training set to be too small, to cause overfitting. Additionally, we use 20% for the test set this time instead of 10%, because we already partitioned 90% of 'df_new' for grade, and partitioning another 10% could end up causing 'test_set' to be too small to generate accurate predictions.

```
## PARTITIONING DATA INTO TRAINING AND TEST AGAIN
set.seed(1)
test_index <- createDataPartition(grade$G3, times=1, p=0.2, list=FALSE)
test_set <- grade[test_index,]
training_set <- grade[-test_index,] #80% of the data used for the training set
```

We will define the Random Mean Squared Error (RMSE) function. This is the value by which we will assess how accurate our predictions are.

Here is the formula for RMSE in R code:

```
rmse <- function(actual, predicted){
  sqrt(mean((actual - predicted)^2))
}
```

Moving on with our model, the formula for a baseline linear model is the following:

$Y_{u,i} = \mu + \epsilon_{u,i}$

We will create a baseline prediction using the mean G3 score to use as a reference for when we later add biases and start regularization. The mean G3 score can be given by $\mu + \epsilon_{u,i}$.

We will begin by calculating μ:

```
mu <- mean(training_set$G3) ## represents the global score
mu ## mu is 11.55469
```

```
## [1] 11.55469
```

μ is 11.55469. We will use this in our baseline prediction and see the accuracy:

```
rmse(mu, training_set$G3) ## 3.182738
```

```
## [1] 3.182738
```

The RMSE is about 3.182738. This is relatively high, we are about three points off on accuracy by just using μ. We will account for added biases from G1 and G2 in order to lower the RMSE

Bias

The following is the mathematical formula for a baseline linear model with added biases:

$\hat{Y}_{u,i} = \mu + b_i + \epsilon_{i,u}$

This is the formula to mathematically calculate predictor's bias:

$\hat{b}_i = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{\mu})$

Knowledge of the above formulas was inspired from information within several courses from the HarvardX Data Science Program.

We can write these function into R code with the following:

```
## Creating a function to get predictor's bias
bias <- function(column_name){
  training_set %>%
    group_by(!!sym(column_name)) %>%
    summarize(bias=mean(G3 - mu, na.rm = TRUE))
}
```

Since we now have a function that can predict the bias for different predictors, we will create a prediction function that will use the formula for the linear model with biases to make our model predictions. That can be done with the following code:

```r
## Creating a prediction function now that we have the bias
predicted <- function(bias, column_name){
  test_set %>%
    left_join(bias, by = column_name) %>%
    mutate(prediction = mu + replace_na(bias,0)) %>%
    pull(prediction)
}
```

Now we can use the following code to calculate the RMSE value using the bias and prediction functions.

That can be done with this following code.

```r
grade_rmse <- function(column_name){
  bias <- bias(column_name)
  predicted <- predicted(bias, column_name)
  rmse(predicted,test_set$G3)
}
```

We get the RMSE from each predictor and see if they had any effect on lowering the RMSE of the model.

```r
grade_rmse('G1') ## 1.611524
```

```
## [1] 1.611524
```

```r
grade_rmse('G2') ## 0.8607571
```

```
## [1] 0.8607571
```

```r
## both heavily improved RMSE
```

Both G1 and G2 heavily lowered the RMSE and made the model extremely more accurate. Seeing that they are both effective, we will combine the biases and use them in our model. Combining biases rather than just individual biases will make our model even stronger.

The formula for combined biases is the following:

$\hat{b}_u = \frac{1}{N}\sum_{i=1}^{N}\left(y_{u,i} - \hat{b}_i - \hat{\mu}\right)$

Again, we will translate the mathematical formula and write it into R code with the following and combine the biases:

```r
G1_bias <- bias('G1')
G1_G2_bias <- training_set %>%
  left_join(G1_bias, by = 'G1') %>%
  group_by(G2) %>%
  summarize(G2_bias=mean(G3-mu-bias, na.rm = TRUE))
```

We have combined our biases from G1 and G2. While we're combining the biases we must also stay wary of NA values that occur during this process and get rid of them.

Next we will use the formula for combined biases within a linear model to create our prediction functions and get our RMSE.

The mathematical formula for combined biases within a linear model can be given by:

$\hat{Y}_{u,i,g} = \mu + b_i + b_u + b_g + \epsilon_{i,u}$

We will, once again, translate this and write it into R code to perform our prediction.

```
prediction <- test_set %>%
  left_join(G1_bias, by = 'G1') %>%
  left_join(G1_G2_bias, by = 'G2') %>%
  mutate(prediction=mu+bias+G2_bias) %>%
  pull(prediction)
rmse(prediction, test_set$G3) ## 1.485441
```

```
## [1] 1.485441
```

Our RMSE greatly improved to 1.485441, a drastic difference from our original RMSE from μ of 3.182738.

Although we have already drastically lowered the RMSE, we will do Regularization in order to get an even more accurate prediction.

Regularization

Information, knowledge, and formulas regarding Regularization had minimal inspiration from HarvardX Data Science Program: Machine Learning.

The process of regularization will assign a penalty λ to the model, which will make our prediction more accurate, and also make sure that we are not overfitting. The penalty λ will fix errors within the model and make sure that we do not have overfitting

The formula for regularization is the following:

$\hat{b}_u = \frac{1}{n_u+\lambda} \sum_{i=1}^{n_u} (y_{u,i} - \hat{\mu})$

The first step to adding regularization to our model is finding the best λ or penalty for our model. We will create a sequence of λ and find the RMSE for each. The λ that gives the smallest RMSE is the best for our model, because a smaller RMSE means that the model is more accurate.

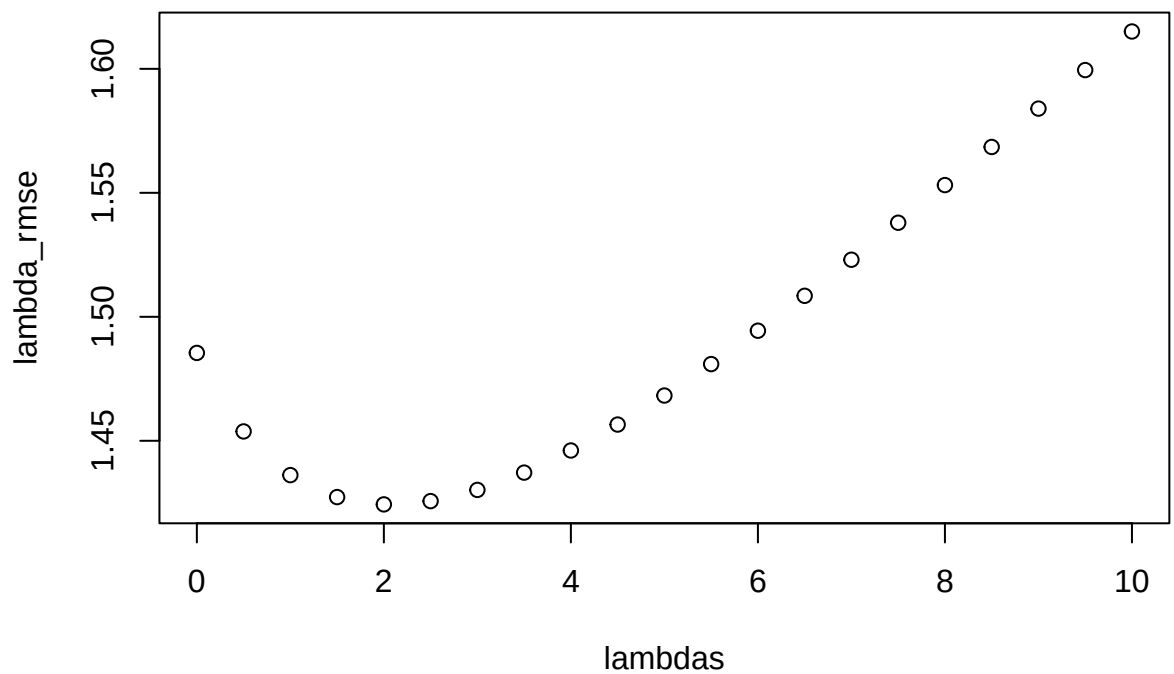
```
## Use regularization
lambdas <- seq(0,10,0.5)
lambda_rmse <- sapply(lambdas, function(lambda){
  G1_bias <- training_set %>%
    group_by(G1) %>%
    summarize(bias = sum(G3-mu)/(n()+lambda))

  G2_bias <- training_set %>%
    left_join(G1_bias, by = "G1") %>%
    group_by(G2) %>%
    summarize(G2_bias = sum(G3-mu-bias)/(n()+lambda))

  prediction <- test_set %>%
    left_join(G1_bias, by = 'G1') %>%
    left_join(G2_bias, by = 'G2') %>%
    mutate(
      bias = replace_na(bias,0),
      G2_bias = replace_na(G2_bias,0),
      prediction = mu+bias+G2_bias) %>%
```

```
    pull(prediction)
  rmse(prediction,test_set$G3)
})

plot(lambdas, lambda_rmse)
```




```
## Calculating best lambda
lambda <- lambdas[which.min(lambda_rmse)]
lambda ## best lambda is 2
```

```
## [1] 2
```

After calculating RMSE's for a sequence of λ and plotting them, we can see that the lowest RMSE is at 2, which means that our model is the most accurate when the penalty λ is 2.

Final Test and Result

Since we have now trained our model with a baseline linear model with added combined biases and regularization, finding the best λ, we will now do our final test on 'final_test' our training set here is going to be 'grade'.

```
## Final RMSE Calculation
final_mu <- mean(grade$G3)
```

```r
## Final bias Calculations
final_G1_bias <- grade %>%
  group_by(G1) %>%
  summarize(bias = sum(G3-final_mu)/(n()+lambda))

final_G1_G2_bias <- grade %>%
  left_join(final_G1_bias, by = 'G1') %>%
  group_by(G2) %>%
  summarize(G2_bias=sum(G3 - final_mu - bias)/(n()+lambda))

##Final Prediction
final_prediction <- final_test %>%
  left_join(final_G1_bias, by = 'G1') %>%
  left_join(final_G1_G2_bias, by = 'G2') %>%
  mutate(
    bias=replace_na(bias, 0),
    G2_bias = replace_na(G2_bias,0),
    prediction = final_mu+bias+G2_bias) %>%
  pull(prediction)
```

And our final RMSE for this model:

```r
rmse(final_prediction,final_test$G3) #Final RMSE is 1.138002
```

```
## [1] 1.138002
```

Our final RMSE is 1.138002, another significant improvement from our original and most previous RMSE. Using a linear model with the combined predictor's biases of G1 and G2, and Regularization, we have built a model that can predict a student's G3 score within approximately 1.14 points.

Random Forest Model

Although we have created a very accurate model for predicting G3 scores using G1 and G2 as predictors, the prediction might be seen as "useless". It makes common sense that you can accurate guess a student's final year score based on what he got during the first and second periods. Therefore, building a model with only those predictors could be seen as not that insightful, although being extremely accurate.

Because of that, we will construct a second model containing all the predictors excluding G1 and G2. Using predictors that aren't clearly as correlated to G3 as G1 and G2, could provide new insights on whether they can be used as an accurate measure for G3 scores. Looking back at the correlation chart created in Data Analysis, none of the predictors seemed very highly correlated with G3, so it is very likely we will achieve a lower RMSE than with the previous model.

To do this, we will be using a Random Forest model. A Random Forest model here is ideal because it works excellently with different data types. Within the data set we have numerical, binary, and categorical variables. We also have the ability to use different metrics like finding the best 'mtry' and cross-validation to prevent overfitting.

We must begin by creating our new datasets since we do not want to have the predictors G1 and G2. We will take our 'grade' training set and 'final_test' test set from previous steps, and remove the predictors G1 and G2.

The following code will do that for us:

```r
## The structure for the random forest models were inspired from examples in the HarvardX Data Science
## Random Forest Model WITHOUT G1 and G2
## Get rid of highly correlated variables
highly_correlated <- c("G1", "G2")
grade_new <- grade %>%
  select(-all_of(highly_correlated)) ## minor inspiration from GitHub Copilot
final_test_new <- final_test %>%
  select(-all_of(highly_correlated)) ## minor inspiration from GitHub Copilot
```

Our new training set is 'grade_new' and our new testing set is 'final_test_new'.

We will train a Random Forest model now, implementing cross-validation and finding the best 'mtry' value using a sequence of 'mtry' values. Cross-validation and finding the best 'mtry' will help prevent overfitting.

Here is the code:

```r
## Set up Random Forest Training Model
rf_model <- train(
  G3 ~ .,
  data = grade_new,
  method = "rf",
  trControl = trainControl(method = "cv", number = 5),
  tuneGrid = data.frame(mtry = 1:10)
)
```

Now we can find the best 'mtry' value.

```r
best_mtry <- rf_model$bestTune$mtry
best_mtry ## best_mtry is 9
```

```
## [1] 9
```

Now that we have the best 'mtry' value we can use that to test and predict the G3 scores using the following code:

```r
final_rf_model <- randomForest(
  G3 ~ .,
  data = grade_new,
  mtry = best_mtry,
  ntree = 500,
  importance = TRUE
)

rf_predicted <- predict(final_rf_model, newdata = final_test_new)
rf_rmse <- rmse(final_test_new$G3, rf_predicted)
rf_rmse ## RMSE was 3.171029
```

```
## [1] 3.171029
```

Our RMSE is 3.171029. Our model is still able to predict a G3 score of a student within approximately 3 points, but as expected, our RMSE from this model is significantly higher than what we found in our first model. This is because the predictors outside of G1 and G2 do not have much correlation with G3 as seen from the correlation chart.

Attempting to find more "important" predictors and doing Random Forest predictions on them will only increase the RMSE even more. In fact, using any smaller combination of predictors will only heighten our RMSE.

Final Model and Results

It has been made clear that for our models to be accurate we have to take the G1 and G2 score into account. Therefore, for our final model we will attempt to use all predictors, including G1 and G2. This will also see if the added predictors produce that "random noise" we were worried about before making the first model.

Our training and test sets will be the same as the ones used for the first model, 'grade' and 'final_test'.

Here is the code for our final RMSE values.

```
## Now we will do a random forest prediction of all predictors
rf_model_all <- train(
  G3 ~ .,
  data = grade,
  method = "rf",
  trControl = trainControl(method = "cv", number = 5),
  tuneGrid = data.frame(mtry = 1:10)
)

final_best_mtry <- rf_model_all$bestTune$mtry
final_best_mtry ## final_best_mtry is 10
```

```
## [1] 10
```

```
final_rf_model_all <- randomForest(
  G3 ~ .,
  data = grade,
  mtry = final_best_mtry,
  ntree = 500,
  importance = TRUE
)

rf_predicted <- predict(final_rf_model_all, newdata = final_test)
rf_rmse <- rmse(final_test$G3, rf_predicted)
rf_rmse ## RMSE was 0.9568145
```

```
## [1] 0.9568145
```

Our final RMSE is 0.9568145.

Conclusion

Throughout the project we created three different models to try to accurately predict the G3 scores of two Portuguese schools. Our first model was a baseline linear model with added predictor's bias from G1 and G2 and Regularization. The first model was able to predict G3 scores within just over one point. We only used G1 and G2 because we felt that the other predictors would cause random noise, because they did not show much correlation on the correlation chart created in Data Analysis. Our second model was a Random Forest model using all the predictors except for G1 and G2. We did this because using G1 and G2 was not

that insightful. It follows common logic that someone would get a final year score (G3) similar to what they got the first two periods (G1 and G2). Therefore, we tried to see the effect of not using those predictors. The Random Forest model ended up getting an RMSE of just over 3 points. This was much higher than the previous RMSE. Using less predictors to try to get rid of random noise only increased the RMSE. It was clear that G1 and G2 had to be used to be able to make an accurate prediction. Our third and final model was another Random Forest model, but this time we used all predictors. Our final RMSE was just under 1 point, an improvement from both our previous models. There are several insights and impacts that we can see from this report. The weak correlation between the majority of the predictors and the final G3 score could potentially, although unlikely, be used to debunk some ideas that students perform at school based on outside circumstances or details. We were able to best predict the test scores using data from previous test scores. That means empirics were more effective as a prediction measure than present circumstances at home or extracurricular involvement. It could be used to argue in other fields that the best way to prove something or predict something will happen is through empirics. In addition, the accuracy was always higher whenever we had a higher number of predictors. If we tried to take away some predictors, the RMSE would go down. There was no random noise that we were worried about while making the first model. This is seen by how the final model with every single predictor was more accurate than the first model with just G1 and G2. This also means that generally the more factors you put into the prediction model, no matter how little, will always contribute to predicting the outcome effectively.

Some limitations of this model was that predictors outside G1 and G2 did not contribute that much to the accuracy of the model.

In future works, we will have more advanced and detailed data that can correlate more closely to the G3 scores. We might also use different models such as K-nearest neighbors.

References

All the work and code in the report are entirely original. Additionally, none of the code or text in this report was AI generated. If any inspiration or knowledge from outside sources was used, it has been properly cited in code comments, text, or in the References page.

Irizarry, Rafael A.. (n.d.). Introduction to Data Science. Harvard University

https://rafalab.dfci.harvard.edu/dsbook

HarvardX. (2025). PH125.7x. Linear Regression. edX

https://learning.edx.org/course/course-v1:HarvardX+PH125.7x+1T2025/home

HarvardX. (2025). PH125.8x. Machine Learning. edX

https://learning.edx.org/course/course-v1:HarvardX+PH125.8x+1T2025/home

HarvardX. (2025). PH125.9x. Capstone Projects. edX

https://learning.edx.org/course/course-v1:HarvardX+PH125.9x+1T2025/home

UC Irvine Machine Learning Repository. (n.d.). Student Performance Dataset

https://archive.ics.uci.edu/dataset/320/student+performance

Student Performance Dataset Download Link. (n.d.):

https://archive.ics.uci.edu/static/public/320/student+performance.zip

GitHub Copilot. GitHub

https://github.com/copilot